

**SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAŽDIN**

Zlatko Pračić

**IZRADA AGENTA S GENERATIVNOM
UMJETNOM INTELIGENCIJOM TEKSTA
SPECIFIČNE NAMJENE**

DIPLOMSKI RAD

Varaždin, 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ž D I N

Zlatko Pračić

Matični broj: 0016145097

Studij: Baze podataka i baze znanja

**IZRADA AGENTA S GENERATIVNOM UMJETNOM INTELIGENCIJOM
TEKSTA SPECIFIČNE NAMJENE**

DIPLOMSKI RAD

Mentor:

Prof. dr. sc. Markus Schatten

Varaždin, kolovoz 2026.

Izjava o izvornosti

Izjavljujem kako je ovaj diplomski rad izvorni rezultat mog rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autor potvrdio prihvatanjem odredbi u sustavu FOI Radovi

Sažetak

Opsega od 100 do 300 riječi. Sažetak upućuje na temu rada, ukratko se iznosi čime se rad bavi, teorijsko-metodološka polazišta, glavne teze i smjer rada te zaključci.

Ključne riječi: riječ; riječ; riječ; Obuhvaća 7 ± 2 ključna pojma koji su glavni predmet rasprave u radu.

Sadržaj

1. Uvod	1
2. Povijest Chatbotova	2
2.1. Teorijske i matematičke pretpostavke	2
2.1.1. Markovljev lanac	2
2.1.2. Turingov test	3
2.2. Prva generacija - tekstualni chatbotovi bazirani na pravilima	3
2.2.1. ELIZA	3
2.2.2. PARRY	4
2.2.3. Racter	5
2.3. Druga generacija - skriptni jezici	5
2.3.1. Mr. Spock u TINYMUD okruženju	6
2.3.2. ALICE i AIML	6
2.4. Treća generacija - komercijalni i praktični chatboti	6
2.4.1. SmarterChild	6
2.4.2. Web i društvene mreže	7
2.5. Četvrta generacija - glasovni asistenti i konverzacijska sučelja	7
2.5.1. Siri, Google Assistant, Cortana, Alexa	7
2.5.2. Opća arhitektura modernog asistenta - Siri	8
2.6. Peta generacija - duboko učenje, transformeri i LLM chatbotovi	8
2.6.1. Sekvencijski modeli	8
2.6.2. Transformer arhitektura	9
2.6.3. Veliki jezični modeli i generativni chatboti (ChatGPT, Bard ...)	10
3. AI inženjerstvo - koncepti i ključne tehnike	11
3.1. Pojam i uloga AI inženjerstva	11
3.2. Temeljni modeli i veliki jezični modeli (LLM)	11
3.2.1. Temeljni modeli	11
3.2.2. Veliki jezični modeli (LLM)	12
3.3. Tehničke osnove: transformer i <i>attention</i>	13
3.3.1. Transformer arhitektura	13
3.3.2. <i>Attention</i> mehanizam	14
3.4. Rad s LLM-ovima u aplikacijama: tokeni, kontekst i kontrola generiranja	15
3.4.1. Tokeni i tokenizacija	15
3.4.2. Kontekst modela	15
3.4.3. Parametri generiranja: <i>temperature</i> , top-k i top-p	16

3.5. Prilagodba ponašanja bez treniranja: prompt engineering i <i>in-context</i> učenje . . .	17
3.5.1. Prompt engineering	18
3.5.2. Sistemski i korisnički prompt	19
3.5.3. <i>In-context</i> learning	19
3.6. Prilagodba modela treniranjem i optimizacija primjene	20
3.6.1. Fino podešavanje i PEFT	20
3.6.2. Preference fine-tuning	21
3.6.3. Kvantizacija i destilacija	22
3.7. RAG: povezivanje modela s vanjskim znanjem	22
3.7.1. Osnovna ideja RAG-a	23
3.7.2. Embeddings i vektorske baze podataka	23
3.7.3. Segmentacija dokumenata - <i>Chunking</i>	24
3.8. GraphRAG	24
4. Implementacija chatbota	25
4.1. Opis domene i zahtjevi	25
4.2. Arhitektura sustava	25
4.3. Korišteni modeli i alati	25
4.4. Korpus i priprema podataka	25
5. Evaluacija chatbotova	26
5.1. Operativni aspekti: inferenca i latencija	26
5.1.1. Online i batch inferenca	26
5.1.2. Latencija	26
5.2. Automatske metrike	27
5.2.1. Leksičke metrike	27
5.2.1.1. ROUGE-L	27
5.2.2. Semantičke metrike	27
5.2.2.1. BERTScore	28
5.2.2.2. Kosinusna sličnost (Cosine Similarity)	28
5.2.3. Ograničenja automatskih metrika u pravnoj domeni	28
5.3. Ljudska evaluacija	29
5.3.1. Evaluatori i dimenzije ocjenjivanja	29
5.3.2. Međuocjenjivačka pouzdanost	29
5.4. LLM-as-a-Judge	30
5.4.1. Pristup i motivacija	30
5.4.2. Implementacija i dizajn prompta	30
5.4.3. Poznata ograničenja	30
5.5. Obrazloženje odabira kombinacije metoda	31
6. Statistička analiza rezultata evaluacije	32
6.1. Pregled prikupljenih podataka	32
6.2. Usporedba konfiguracija: Vanilla vs. RAG vs. GraphRAG	32

6.3. Korelacija automatskih metrika s ljudskom evaluacijom	33
6.4. Međuocjenjivačka pouzdanost	34
6.5. LLM-as-a-Judge: usporedba s ljudskom evaluacijom	34
6.6. Latencija	34
6.7. Sinteza rezultata	34
7. Zapažanja tijekom razvoja Chatbota	35
7.1. Evolucija RAG sustava kroz iterativno poboljšavanje	35
7.2. Specifični tehnički izazovi i njihova rješenja	35
7.3. Optimizacija generacijskih parametara i njihov utjecaj	35
7.4. Zaključak i naučene lekcije	36
8. Zaključak	37
Popis literature	39
Popis slika	40
Popis tablica	41
1. Prilog 1	43

1. Uvod

"We are drowning in information but starved for knowledge." [1]
– John Naisbitt, Megatrends

2. Povijest Chatbotova

Popularnost chatbotova naglo je porasla predstavljanjem ChatGPT-a 2022. godine. Iz te se činjenice može zaključiti kako su chatbotovi postojali i prije ove nagle popularizacije. Stoga će se u ovom poglavlju prikazati povijesni razvoj chatbota, kao i teorijska osnova koja je utjecala na razvoj ideje o stroju koji razgovara.

2.1. Teorijske i matematičke pretpostavke

Iako je Andrey Markov uvođenjem Markovljevih lanaca u svom radu postavio važan temelj, ideja njegovog rada nije bila ni približna konceptu “stroja koji razgovara”. Vrijednost modela Markovljevog lanca za kasnija otkrića i razvoj znanstvene misli je neupitna.

Alan Turing promišlja o budućnosti strojeva čije su se sposobnosti ubrzano razvijale i postajale sve složenije. Zanimalo ga je može li stroj tijekom komunikacije uvjeriti ispitivača da nije stroj, što se u nastavku teksta formalizira u obliku Turingova testa. Iako se u tom kontekstu izravno ne spominju chatbotovi, upravo se na njih ovaj test u velikoj mjeri može primijeniti.

2.1.1. Markovljev lanac

Al-Amin i ostali u svom radu predstavljaju Markovljev lanac kao jednu od najranijih metoda koja je pokazala potencijal chatbota kroz generiranje teksta na temelju statističkih vjerojatnosti slijeda riječi. Markovljev lanac je razvio Andrey Markov 1906. godine za modeliranje stohastičkih procesa, a kasnije se počeo koristiti u zadacima poput predviđanja sljedeće riječi (npr. *autocomplete* u e-pošti). U chatbot kontekstu, Markovljev lanac procjenjuje vjerojatnost da određena riječ slijedi drugu i na temelju unaprijed naučenih prijelaznih vjerojatnosti “slaže” odgovor. Kao rana primjena navodi se HeX, koji je osvojio Loebnerovu nagradu 1997., nakon čega je interes za ovu tehniku postupno rastao [2].

Klasični Markovljevi modeli opisuju prelazak sustava između različitih stanja kroz vrijeme, pri čemu se prijelazne vjerojatnosti procjenjuju iz empirijskih podataka i pretpostavlja se da su stabilne kroz susjedne vremenske intervale. Prostorni Markovljevi modeli proširuju tu ideju na prostorne jedinice (npr. namjene zemljišta) te uključuju prostornu ovisnost između susjednih lokacija, a dodatna proširenja omogućila su i višedimenzionalne simulacije [2].

Unatoč jednostavnosti implementacije, probabilistički pristup zasnovan na Markovljevim lancima pati od ozbiljnih ograničenja u kontekstu prirodnog dijaloga. Budući da modeli predviđanja obuhvaćaju tek neposredno prethodne tokene (jednu ili nekoliko riječi), izostanak šireg kontekstualnog prozora onemogućuje zadržavanje dugoročne strukture rečenice i smisla razgovora. Ova je tehnika povijesno redefinirala pristup obradi jezika, trasirajući put od strogo determinističkih pravila prema generativnim modelima [2].

2.1.2. Turingov test

Alan Turing svojim radom tvrdi kako je pitanje “Mogu li strojevi misliti?” nejasno pa ga zamjenjuje praktičnijim kriterijem: imitacijskom igrom (u nastavku kaže kako je to “test”), gdje ispitivač preko pisanih poruka pokušava razlikovati čovjeka od stroja. Fokusira se na digitalna računala, objašnjava njihovu građu (memorija, izvršna jedinica, kontrola/program) i naglašava njihovu univerzalnost. Jedno dovoljno snažno računalo može oponašati ponašanje bilo kojeg stroja s diskretnim stanjima ako je dobro programirano. Predviđa kako bi se u nekoliko desetljeća mogla pojaviti računala koja bi u takvoj igri bila dovoljno uvjerljiva u “razgovoru” s prosječnim ispitivačem [3].

Velik dio rada posvećen je odgovorima na prigovore kako strojevi ne mogu misliti: teološke tvrdnje o “duši”, strah od posljedica, matematička ograničenja, argument iz svijesti i razne “nemogućnosti” tipa da stroj ne može biti kreativan, griješiti ili učiti. Turing uglavnom pokazuje da ti prigovori ne dokazuju nemogućnost, često miješaju kvarove s pogrešnim zaključivanjem ili se oslanjaju na tadašnja ograničenja tehnologije. Kao pozitivni smjer predlaže da se ne programira “odrasli um” direktno, nego da se napravi “dječji stroj” koji se zatim uči i odgaja (nagrade/kazne, jezik naredbi, moguće i slučajnost), analogno procesu učenja i evolucije [3].

Al-Amin kaže kako se Turingov test može smatrati modelskom pretečom chatbota jer je već 1950. postavio razgovor u prirodnom jeziku kao ključni način provjere “inteligentnog” ponašanja mašine, kroz dijalog u kojem treba biti teško razlikovati mašinu od čovjeka. Evaluacija se zasniva na tome može li računalo “prevariti” ljudskog ispitivača da pomisli kako razgovara s čovjekom, što je isti osnovni cilj konverzacijskih sistema poput chatbota. Također, kao utjecajan istraživački kamen temeljac, Turingov test je godinama i desetljećima motivirao razvoj obrade prirodnog jezika, čime je direktno usmjerio put prema modernim chatbot tehnologijama [2].

2.2. Prva generacija - tekstualni chatbotovi bazirani na pravilima

Prva generacija chatbotova pojavila se između sredine 1960-ih i ranih 1980-ih godina. Tada su programeri prvi put pokušali svoje teorijske ideje prevesti u stvarne razgovore. Ovi su sustavi tehnički funkcionirali tako što su prepoznavali unaprijed definirane tekstualne obrasce i primjenjivali strogo definirana, ručno napisana pravila. Nisu stvarno razumjeli što tekstovi znače ili u kojem kontekstu stoje. Sustav je tražio važne riječi u poruci osobe koja ga koristi i odgovarao unaprijed definiranim frazama. To je zvučalo kao pravi razgovor, ali zapravo nije bilo. Unutar usko definiranih područja, ovi su chatbotovi postavili temelje za to kako ljudi i računala komuniciraju. Također su pokazali da ljudi često sklonost imaju prema tome da strojevima daju ljudske osobine.

2.2.1. ELIZA

Rupert i ostali u svom radu detaljno obrađuju ELIZA, chatbot kojeg je Joseph Weizenbaum razvio na MIT-u sredinom 1960-ih godina. Po tom istraživanju, ELIZA predstavlja prvi

```

EEEEEE LL      IIII ZZZZZZ  AAAA
EE      LL      II      ZZ  AA  AA
EEEEEE LL      II      ZZZ  AAAAAA
EE      LL      II      ZZ  AA  AA
EEEEEE LLLLLL IIII ZZZZZZ  AA  AA

Eliza is a mock Rogerian psychotherapist.
The original program was described by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

ELIZA: Is something troubling you ?
YOU:   Yes. I'm trying to describe you for a research paper I'm writing. What sh
ould I say?
ELIZA: You are sure.
YOU:   Yes, I'm sure.
ELIZA: I see.
YOU:   What can I tell them about you?
ELIZA: You seem to need to tell them about me.
YOU:   I absolutely need to. Please help.
ELIZA: That's quite interesting.
YOU:   I agree.
ELIZA: You say you agree ?
YOU:

```

Slika 1: Originalni ispis koda ELIZA pronađen u Weizenbaumovim arhivama [5]

chatbot na svijetu, nastao na prvom *time-sharing* sustavu CTSS (*Compatible Time-Sharing System*) na IBM 7094 računalu. Rad opisuje kako je tim istraživača 2021. otkrio originalni ispis koda ELIZA u Weizenbaumovim arhivama na MIT-u, uključujući ranu verziju čuvenog DOCTOR skripta te gotovo potpunu verziju MAD-SLIP koda, kombinacije programskog jezika MAD i Weizenbaumovog vlastitog sustava SLIP (*Symmetric Lisp Processor*). Tim je uspješno “vratio na život” originalnu ELIZA na emuliranom IBM 7094 sustavu u prosincu 2024., čime je po prvi put nakon više od 60 godina omogućeno pokretanje autentičnog originalnog koda. Bit ovog postignuća nije samo nostalgija, rekonstrukcija je omogućila provjeru što originalna ELIZA doista jest, potvrđujući kako je bila funkcionalan, iako ograničen sustav koji je pomogao pokrenuti cijelo područje konverzacijske umjetne inteligencije [4].

Analiza restauriranog koda Ruperta i ostalih otkrila je nekoliko ključnih aspekata koji ranije nisu bili poznati. Pronađena verzija ima implementiran potpuni “*teaching mode*” (način podučavanja), spominjan u Weizenbaumovom radu iz 1966. samo jednom rečenicom, ali nikada detaljno opisan. Korisnici su mogli upisivanjem naredbi poput ADD, SUBST, APPEND, RANK, TYPE, DISPLA i START mijenjati pravila razgovora i spremati nove skripte. Također, otkrivena je značajna greška: ELIZA se “srušila” kada bi primila numerički unos (npr. “*you are 999 today*”), što autori tumače kao ironično doslovno shvaćanje Ada Lovelace uvida da računala mogu “djelovati na druge stvari osim brojeva”. Važno je da tim nije popravio takve bugove, ostavili su kod autentičnim, točno 96% onakav kakav je pronađen, kako bi očuvali arheološku vrijednost nalaza. Time ELIZA postaje ne samo povijesni artefakt već i upozorenje da su mnogi kasniji opisi (npr. da je ELIZA bila napisana u Lispu) bili pogrešni, originalna je bila u MAD-SLIP jeziku, a Lisp verzija nastala je kasnije kao klon Bernieja Cosella, koji se zatim proširio preko ARPANeta [4].

2.2.2. PARRY

Al-Amin u svom radu obrađuju PARRY chatbot koji je 1972. razvio psihijatar Kenneth Colby na Stanfordu i predstavlja suprotnost ELIZI. Umjesto da glumi terapeuta, PARRY glumi

paranoidnog shizofrenog pacijenta kako bi odvuкао pažnju sa sebe i potaknuo korisnika na dugačke, razrađene odgovore. PARRY je služio i kao alat za obuku psihijatarā (vježbanje razgovora s pacijentima), ali i kao operativni model Colbyjeve teorije paranoje, tj. demonstracija kako “poremećena interpretacija znakova” može proizvoditi prepoznatljive obrasce paranoidnog govora i ponašanja [2].

PARRY je jedan od prvih chatbotova koji se evaluirao kroz oblik “Turing testa” (imitacijska igra) radi procjene koliko uvjerljivo može oponašati ljudski razgovor. Ta linija evaluacije kasnije se nastavlja kroz natjecanja poput Loebner Prizea, gdje se botovi ocjenjuju po “prirodnosti” nakon kratkih razgovora. Poanta u tekstu je da su PARRY (i kasnije Loebner Prize) dio ranih pokušaja mjerenja konverzijske “inteligencije” chatbotova prema tome koliko dobro simuliraju prirodan ljudski dijalog [2].

2.2.3. Racter

Racter (1983), kojeg su izradili Chamberlain i Etter, u tekstu se opisuje kao pionirski chatbot koji se posebno istaknuo po nasumičnom generiranju novog teksta, a ne samo biranju odgovora iz predložaka. Racter (skraćeno od *raconteur*, pripovjedač) mogao je tijekom razgovora stvarati jedinstvene rečenice i prozne ulomke koristeći pravila kontekstno-slobodne gramatike (*context-free grammar*) i element slučajnosti. Rad naglašava kako je rezultat često bio besmislen jer nije postojalo “pravo razumijevanje”, ali da je sama proceduralna sposobnost generiranja originalnog teksta bila iznimno napredna i utjecajna za to vrijeme [2].

Posebno se ističe kako je autor povezan s Racterom (Chamberlain) objavio knjigu *The Policeman's Beard is Half Constructed* (1984), koja se predstavlja kao knjiga “autorski” proizvedena Racterovim generativnim mogućnostima, čime je dobio širu kulturnu vidljivost. Tekst napominje kako nije potpuno jasno u kojoj je mjeri proza doista bila u potpunosti računalno generirana, no važna je poruka da je Racter popularizirao ideju chatbotova koji mogu proizvoditi nov i originalan razgovorni sadržaj. Rad daje primjer kako pojedini Racterovi fragmenti zvuče introspektivno i “ljudski” (npr. o snovima i željama), što je otvorilo pitanja o humanizaciji i o tome kako AI može imitirati emocionalni i egzistencijalni izraz iako nema ljudsko iskustvo [2].

2.3. Druga generacija - skriptni jezici

Nakon pionirskih sustava poput ELIZA, razvoj chatbotova ušao je u fazu dominacije skriptnih jezika i naprednog uparivanja uzoraka, što se može smatrati njihovom drugom generacijom. Umjesto bazičnih pravila prve generacije, ovi sustavi uvode strukturirane skriptne jezike poput AIML-a, RiveScripta i ChatScripta. Njihova primjena omogućila je izgradnju opsežnijih baza znanja, čime je olakšano prepoznavanje konteksta i upravljanje kompleksnijim varijacijama korisničkih upita. Iako su se i dalje temeljili na predefiniranim predlošcima bez elemenata strojnog učenja, ovi su jezici značajno podigli prag interaktivnosti [6].

2.3.1. Mr. Spock u TINYMUD okruženju

Godine 1989. James Aspnes sa Carnegie Mellon University razvio je TINYMUD (*multi-player real-time virtual world*), jedan od prvih višekorisničkih tekstualnih virtualnih svjetova. Na tom sustavu pojavio se jedan od prvih chatbot likova, Mr. Spock, koji je mogao odgovarati na osnovna konverzacijska pitanja korisnika koristeći jednostavno podudaranje uzoraka [2].

Mr. Spock predstavljao je rani pokušaj stvaranja umjetnog lika koji može razgovarati s ljudima unutar virtualnog okruženja te je prethodio pojmu NPC (*non-player character*) botova koji su danas uobičajeni u igrama i virtualnim svjetovima [2].

2.3.2. ALICE i AIML

Sljedeći veći korak u povijesti chatbotova je ALICE (*Artificial Linguistic Internet Computer Entity*), nastao 1995. kao *online* chatbot inspiriran ELIZA-om. ALICE je bio temeljen na slaganju uzoraka, bez stvarnog razumijevanja cjelokupnog razgovora [2], [6].

Iako je kasnije poboljšavan i osvojio Loebner Prize za “najhumaniji” računalni program tri puta (2000., 2001. i 2004.), ALICE nije uspio proći puni Turingov test [2].

Eleni naglašava kako je ključna razlika između ALICE i ELIZA-e to što je ALICE razvijen uz novi jezik napravljen upravo za tu svrhu: AIML (*Artificial Intelligence Markup Language*), koji se temelji na XML-u i organiziran je oko kategorija koje se sastoje od uzorka i predloška [6].

Autori kažu kako je AIML nastao u razdoblju 1995–2000 za izgradnju baze znanja chatbotova koji koriste pristup slaganja uzoraka. AIML je baziran na XML-u, a temeljna jedinica znanja je kategorija koja se sastoji od uzorka (*pattern* — korisnički unos) i predloška (*template* — odgovor bota) [6]. Kategorije mogu biti grupirane unutar tema (*topics*), a sve se organizira u strukturu nazvanu Graphmaster koja omogućuje učinkovito uparivanje uzoraka pretraživanjem u dubinu (*depth-first search*) [7].

2.4. Treća generacija - komercijalni i praktični chatboti

Chatbotovi su prošli dug put, od Weizenbaumove ELIZA do asistenata ugrađenih u svaki pametni telefon. Ključni trenutak u toj evoluciji bio je prijelaz s izoliranih programa na masovne komunikacijske platforme, čime su chatbotovi postali dostupni širokom krugu korisnika u stvarnom vremenu. Kroz primjere poput pionirskog SmarterChilda i suvremenih rješenja na platformama kao što su Facebook Messenger, WhatsApp i WeChat, može se pratiti kako su ovi sustavi transformirali način na koji se dobivaju informacije, komunicira s brendovima i koriste društvene mreže [2] [6].

2.4.1. SmarterChild

SmarterChild je chatbot koji je razvila tvrtka ActiveBuddy 2001. godine i smatra se jednim od ranih pionira koji je chatbotove približio širokoj publici. Al-Amin u svom radu navodi

kako je chatbot radio unutar *instant messaging* platformi i mogao je davati informacije o raznim temama na upit, što je pokazalo kako chatbot može funkcionirati kao svojevrsni “virtualni asistent” koji pronalazi činjenice i odgovara na korisnička pitanja [2].

Al-Amin ističe kako je posebno važno da se SmarterChild mogao dodati u MSN Messenger i AOL Instant Messenger te je dosegao više od 30 milijuna korisnika. Time je “pionirski” otvorio put chatbotovima u mrežama za dopisivanje i kasnijim asistentima poput Appleova Siri-ja i Samsungova S Voice-a, jer je demonstrirao potencijal pružanja usluga kroz razgovorno sučelje direktno u okruženju poruka. Također je pokazao praktičnu vrijednost (npr. dohvat vijesti, vremena i burzovnih podataka iz baza) pa je pristup informacijama kroz razgovor umjesto kroz web-stranice označen kao važan korak prema AI sustavima koji služe ljudima putem dijaloga [2].

2.4.2. Web i društvene mreže

Chatbotovi na društvenim mrežama naglo su se proširili paralelno s rastom interneta i društvenih platformi, gdje mogu imati različite uloge: korisničku podršku, marketing i oglašavanje, zabavu te prikupljanje podataka. U radu Al-Amina se također naglašava kako se mogu koristiti i kao instrumenti “hibridnih prijetnji” s ciljem utjecaja na javno mnijenje, što pokazuje koliko su njihove primjene višestruke i društveno značajne [2].

U praksi se chatbotovi često integriraju izravno u brendove profile na društvenim mrežama kako bi pružali trenutnu podršku, automatski generirali sadržaj (npr. opise i hashtagove), omogućili personaliziran “razgovorni” kontakt s publikom, radili analizu sentimenta i prikupljali te organizirali podatke o korisnicima i trendovima. Kao primjer integracije na velikim platformama navodi se Meta, koja uvodi AI značajke te testira “Meta AI” asistenta dostupnog na WhatsAppu, Messengeru i Instagramu, uz naglasak na postupno uvođenje i zaštitne mehanizme [2].

2.5. Četvrta generacija - glasovni asistenti i konverzacijska sučelja

Četvrta generacija razvoja umjetne inteligencije u domeni komunikacije označava prijelaz s tekstualnih chatbotova na inteligentne glasovne asistente integrirane u svakodnevne uređaje. Ova faza, koja je započela početkom 2010-ih, donijela je revoluciju u interakciji čovjeka i računala omogućivši prirodan razgovor, prepoznavanje konteksta i upravljanje pametnim domovima. Moderni asistenti koriste duboko učenje i masovne baze podataka kako bi predvidjeli potrebe korisnika i pružili personalizirano iskustvo u stvarnom vremenu [2] [6].

2.5.1. Siri, Google Assistant, Cortana, Alexa

Pojava Siri na iPhoneu 4S 2011. godine postavila je temelje za komercijalnu upotrebu glasovnih asistenata, proizašavši iz ambicioznog vojnog projekta CALO koji je težio stvaranju kognitivnog agenta sposobnog za učenje. Siri je uvela koncept personaliziranog digitalnog

pomoćnika koji ne samo da odgovara na pitanja, već i izvršava zadatke poput postavljanja podsjetnika ili slanja poruka, prilagođavajući se s vremenom jeziku i preferencijama korisnika. Iako je pionir, njezina ovisnost o internetskoj vezi i ograničena podrška za manje jezike ostali su izazovi u ranoj fazi razvoja [2] [6].

Ubrzo nakon Applea, Google je 2012. predstavio Google Now (kasnije Google Assistant), fokusirajući se na prediktivno pružanje informacija na temelju lokacije i povijesti pretraživanja, dok su Microsoft i Amazon 2014. godine ušli na tržište s Cortanom i Alexom. Dok je Cortana bila usmjerena na produktivnost unutar Windows ekosustava, Alexa je redefinirala koncept pametnog doma kroz Echo uređaje, omogućivši programerima stvaranje novih vještina putem Alexa Skills Kita [2] [6].

2.5.2. Opća arhitektura modernog asistenta - Siri

“Hey Siri” je *on-device* glasovni okidač koji koristi duboku neuronsku mrežu (DNN) za stalno osluškivanje aktivacijske fraze uz minimalnu potrošnju baterije. Sustav na iPhoneu radi u dvije faze: mali DNN na štedljivom koprocesoru (AOP) vrši početnu detekciju, a tek nakon toga glavni procesor s većim modelom potvrđuje upit. Cijeli proces prepoznavanja temelji se na analizi kratkih audio-okvira od 0,2 sekunde i vremenskoj integraciji rezultata kako bi se izračunala pouzdanost okidanja [8].

Nakon lokalne aktivacije, sustav provjerava identitet korisnika usporedbom s glasovnim profilom spremljenim na uređaju. Iako je sam detektor okidača u potpunosti lokalni radi privatnosti i brzine, većina daljnje obrade (glavno prepoznavanje govora i tumačenje upita) odvija se u oblaku. Ako Siri server utvrdi da početak govora nije bio ispravan okidač, šalje signal uređaju da se vrati u stanje mirovanja [8].

2.6. Peta generacija - duboko učenje, transformeri i LLM chatbotovi

Peta generacija uči iz podataka, i to u količinama koje nijedan čovjek ne bi mogao pročitati. Dok su raniji sustavi poput ELIZA-e koristili jednostavno prepoznavanje uzoraka, moderni pristupi temelje se na dubokom učenju i neuronskim mrežama koje simuliraju ljudske kognitivne procese. Ova evolucija omogućila je chatbotima da postanu ne samo digitalni asistenti za izvršavanje zadataka, već i empatični sugovornici sposobni za složenu komunikaciju [6].

2.6.1. Sekvencijski modeli

Sekvencijski modeli, poput rekurentnih neuronskih mreža (RNN) i mreža s dugom kratkoročnom memorijom (LSTM), predstavljali su ključni korak u omogućavanju chatbotima da zadrže kontekst razgovora. Ovi modeli koriste petlje koje omogućuju informacijama da perzistiraju, što je neophodno za razumijevanje rečenica u kojima značenje ovisi o prethodno izgovorenim riječima [9].

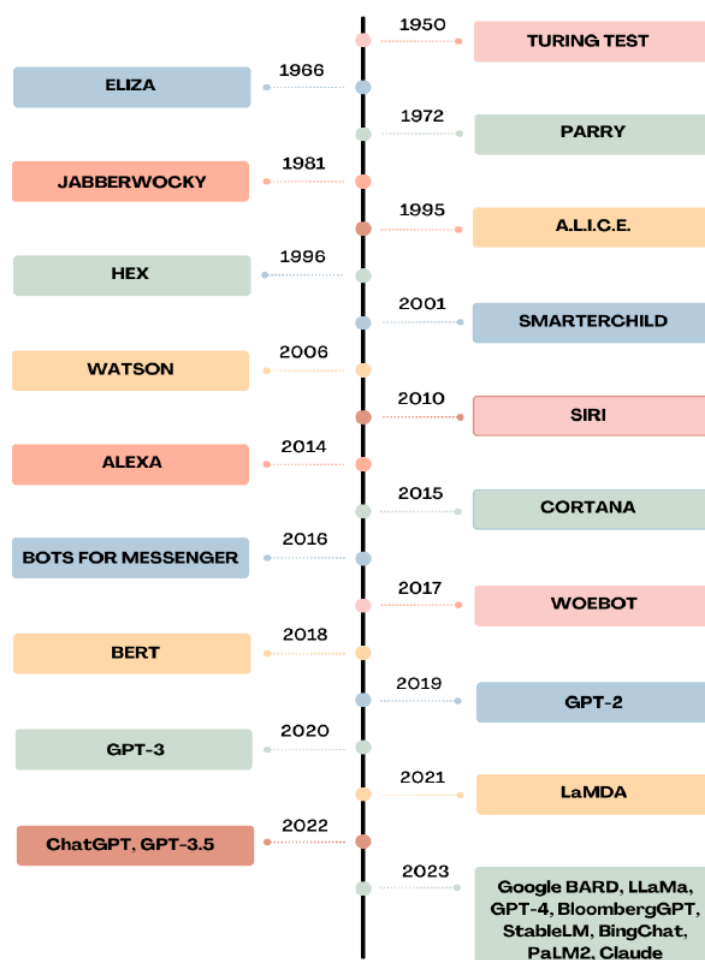
Ovi modeli često nailaze na poteškoće pri učenju dugoročnih ovisnosti u vrlo dugim tekstovima. Iako su LSTM i GRU (*Gated Recurrent Units*) značajno poboljšali preciznost u sustavima za odgovaranje na često postavljana pitanja (FAQ), njihova arhitektura i dalje zahtijeva sekvencijalnu obradu podataka, što ograničava brzinu treniranja na velikim skupovima podataka [9].

2.6.2. Transformer arhitektura

Transformer arhitektura donijela je revoluciju u obradi prirodnog jezika uvođenjem mehanizma “pažnje” (*attention mechanism*), koji omogućuje modelu da istovremeno promatra sve dijelove ulazne sekvence. Transformeri mogu paralelno obrađivati podatke, što ubrzava proces treniranja i omogućuje rad s neusporedivo većim korpusima teksta [10].

Ova arhitektura omogućuje modelu da dinamički dodjeljuje važnost različitim riječima u rečenici, bez obzira na njihovu međusobnu udaljenost, čime se postiže dublje razumijevanje semantike i konteksta. Upravo je ovaj tehnološki skok postavio temelje za razvoj suvremenih velikih jezičnih modela koji dominiraju današnjim tržištem umjetne inteligencije [10].

Na Slici 2 se može vidjeti vremenski slijed pojavljivanja pojedinih chatbotova. Neki od njih nisu spomenuti u ovom radu pošto se u opisu povijesti chatbotova biralo najznačajnije zastupnike.



Slika 2: Vremenska linija chatbota [11]

2.6.3. Veliki jezični modeli i generativni chatboti (ChatGPT, Bard ...)

Veliki jezični modeli (LLM) predstavljaju vrhunac generativne umjetne inteligencije, koristeći milijarde parametara za predviđanje i generiranje teksta koji je često nerazlučiv od ljudskog. Sustavi poput ChatGPT-a ili Barda (sada Gemini) ne oslanjaju se na unaprijed definirane odgovore, već sintetiziraju nove rečenice u stvarnom vremenu, prilagođavajući se tonu i specifičnim zahtjevima korisnika kroz višestruke krugove dijaloga [2].

Iako ovi modeli pokazuju impresivne sposobnosti u rješavanju problema, kreativnom pisanju i programiranju, oni se i dalje suočavaju s izazovima poput "halucinacija" ili generiranja netočnih informacija. Njihova integracija u svakodnevne alate i sposobnost učenja iz povratnih informacija korisnika čine ih najnaprednijim oblikom interakcije čovjeka i stroja do danas [12].

3. AI inženjerstvo - koncepti i ključne tehnike

Ovaj rad bavi se izgradnjom chatbota koji se oslanja na postojeće jezične modele, što ga smješta u područje AI inženjerstva, a ne razvoja modela od nule. U ovom poglavlju objašnjavaju se koncepti koji su bili izravno relevantni za taj proces.

3.1. Pojam i uloga AI inženjerstva

AI inženjerstvo je specijalizirano područje u kojem stručnjaci koriste umjetnu inteligenciju i strojno učenje za razvoj aplikacija i sustava koji organizacijama pomažu povećati učinkovitost, smanjiti troškove i donositi bolje poslovne odluke. AI inženjeri razvijaju alate, sustave i procese koji omogućuju primjenu umjetne inteligencije u stvarnom svijetu [13].

U praksi, AI inženjer najčešće radi na spoju modela i produkcijskog sustava: uzima pretreniran model, prilagođava ga kroz fino podešavanje (*fine-tuning*) ili prompt engineering, i izlaže ga kao API koji ostatak aplikacije može konzumirati. Izgradnja modela od nule danas je rjeđa, i skuplja, nego što je to slučaj s inženjering radom oko postojećih. Potrebne vještine uključuju programiranje u jezicima poput Pythona, R-a, Jave i C++-a, uz solidno poznavanje statistike i linearne algebre. Od infrastrukturnih alata očekuje se iskustvo s *big data* tehnologijama kao što su Apache Spark, Hadoop i MongoDB. Ključan je i praktičan rad s algoritmima strojnog učenja i dubokog učenja, a posebno se cijeni poznavanje popularnih AI *frameworka* poput TensorFlowa, PyTorch-a i Kerasa [13].

3.2. Temeljni modeli i veliki jezični modeli (LLM)

Suvremena paradigma u području umjetne inteligencije označava pomak od razvoja izoliranih, usko specijaliziranih modela prema primjeni pretreniranih sustava opće namjene. U središtu ove paradigme nalaze se temeljni modeli, unutar kojih se veliki jezični modeli (LLM) izdvajaju kao specifična podskupina. Dok su temeljni modeli multimodalni i obuhvaćaju domene poput računalnog vida i obrade zvuka, LLM-ovi su usmjereni na tekstualne podatke, demonstrirajući napredne sposobnosti semantičkog zaključivanja i kontekstualne adaptacije. Razlika između tih dviju skupina nije samo terminološka, ona određuje što od modela možeš tražiti i što možeš očekivati [12], [14].

3.2.1. Temeljni modeli

Temeljni modeli (često u literaturi nazivani i *Pretrained Foundation Models* - PFMs) definiraju se kao modeli velikih kapaciteta koji su pretrenirani na masivnim i heterogenim skupovima podataka, čime stječu opće reprezentacije primjenjive na niz različitih izvodnih zadataka. Povijesni razvoj ovih modela, od ranih jezičnih reprezentacija poput BERT-a do kompleksnih multimodalnih sustava, ukazuje na trend u kojem se kroz povećanje računalne skale i količine podataka postiže visoka razina transfernog učenja. Ovakav pristup omogućuje da se jedan

centralni model, uz minimalne prilagodbe, uspješno koristi u domenama koje variraju od računalnog vida do analize grafova, čime se mijenja proces dizajna inteligentnih sustava [14].

Ključna prednost temeljnih modela leži u njihovoj sposobnosti prilagodbe specifičnim kontekstima putem metoda kao što su fino podešavanje, instrukcijsko učenje ili tehnike parametarski učinkovite adaptacije. Oslanjanje na jedan centralni model donosi i značajne izazove, prvenstveno u pogledu nasljeđivanja pristranosti iz pretreniranih podataka, sigurnosnih rizika te potrebe za rigoroznom evaluacijom robusnosti. Stoga se suvremena istraživanja ne fokusiraju samo na povećanje performansi, već i na razvoj mehanizama koji osiguravaju privatnost, etičnost i pouzdanost modela u produkcijskim okruženjima, što znači da se uz tehničke probleme javljaju i pitanja pristranosti, privatnosti i etike [12].

3.2.2. Veliki jezični modeli (LLM)

Veliki jezični modeli predstavljaju specijalizaciju temeljnih modela unutar domene prirodnog jezika, karakterizirani milijardama parametara i treniranjem na korpusima teksta koji obuhvaćaju značajan dio ljudskog znanja. LLM-ovi pokazuju takozvana “izranjajuća” (emergentna) svojstva – sposobnosti poput rješavanja logičkih problema ili učenja iz konteksta koje se pojavljuju tek pri dosezanju određene kritične mase parametara i podataka. Evolucija ovih modela dovela je do pomaka s jednostavnog predviđanja sljedećeg tokena prema sustavima koji su usklađeni s ljudskim namjerama putem metoda poravnanja, čime postaju sposobni za kompleksnu interakciju i izvršavanje složenih korisničkih uputa [12], [14].

Arhitektonski temelj LLM-ova čini Transformer arhitektura, čija skalabilnost i paralelizacija omogućuju obradu ogromnih količina informacija uz korištenje naprednih varijanti mehanizama pažnje. Kako bi se prevladali visoki računalni troškovi treniranja i inferencije, istraživački fokus se sve više usmjerava prema optimizaciji arhitekture, uključujući efikasnije metode pozicijskog kodiranja i tehnike kompresije modela poput kvantizacije i “prorjeđivanja”. Ovi tehnički napredci omogućuju širu dostupnost LLM-ova, pretvarajući ih iz isključivo istraživačkih artefakata u praktične komponente koje se mogu integrirati u različite softverske arhitekture uz zadržavanje visoke razine preciznosti [14].

U suvremenim primjenama, LLM-ovi se rijetko koriste kao izolirani sustavi. Umjesto toga, oni postaju jezgra širih okvira koji uključuju vanjske izvore znanja kroz metode poput generiranja potpomognutog dohvatom (*Retrieval-Augmented Generation* - RAG). Ovakva integracija rješava inherentne probleme modela, poput halucinacija i nedostatka ažurnih informacija, dok istovremeno omogućuje specijalizaciju za specifične domene bez potrebe za skupim ponovnim treniranjem. Konačno, proces evaluacije LLM-ova evoluirao je od jednostavnih metričkih pokazatelja prema kompleksnim *benchmark* testovima koji ispituju kognitivne sposobnosti, etičku usklađenost i praktičnu korisnost [14] [12].

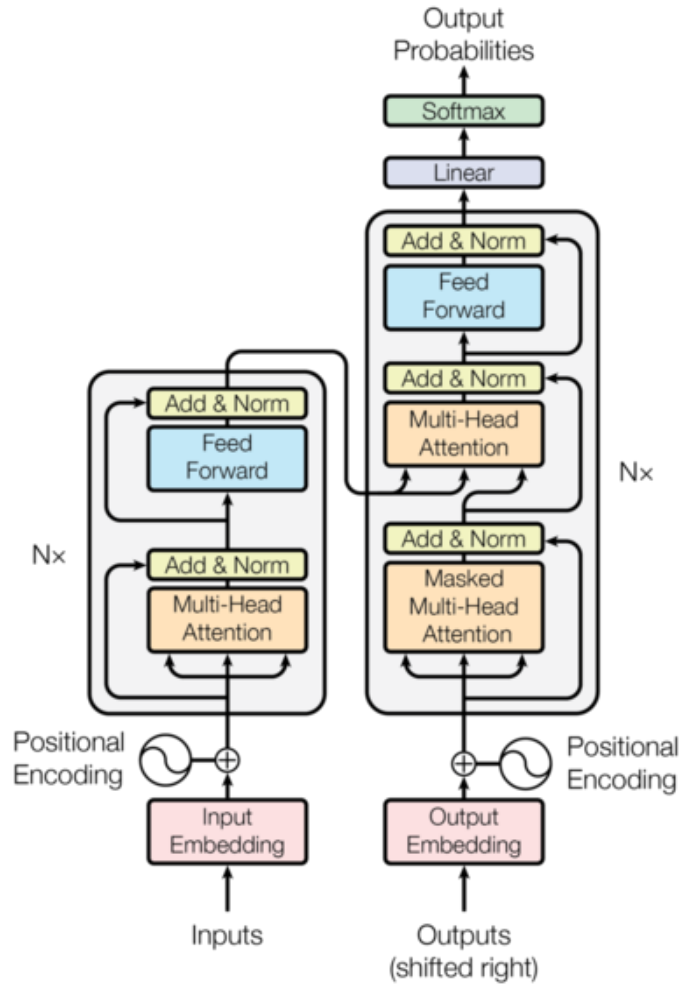
3.3. Tehničke osnove: transformer i *attention*

Prije pojave Transformera, rekurentne neuronske mreže poput LSTM i GRU bile su standardni izbor za obradu sekvenci teksta. Njihov glavni nedostatak bila je sekvencijalna obrada – svaki element sekvence morao se obraditi tek nakon prethodnog, što je onemogućavalo paralelizaciju i usporavalo treniranje na dugim sekvencama [10]. Transformer arhitektura riješila je ovaj problem potpunim odbacivanjem rekurencije i oslanjanjem isključivo na mehanizme pažnje (*attention*), čime je postala temeljna arhitektura za suvremene jezične modele [14].

3.3.1. Transformer arhitektura

Transformer se sastoji od dva glavna dijela: koder (*encoder*) koji obrađuje ulazni tekst i dekoder (*decoder*) koji generira izlazni tekst. Koder pretvara ulaznu sekvencu u numeričke reprezentacije koje dekoder koristi za generiranje izlaza, element po element. Model je autoregresivan – pri generiranju svakog novog elementa koristi sve prethodno generirane elemente kao kontekst [10].

Ključna prednost ove arhitekture je sposobnost da istovremeno “vidi” sve dijelove ulazne sekvence, neovisno o njihovoj međusobnoj udaljenosti. Ovo omogućuje učinkovito hvatanje dugodosežnih ovisnosti u tekstu, što je kod rekurentnih mreža predstavljalo značajan izazov [10]. Također, budući da se sve pozicije mogu obraditi paralelno, Transformer omogućuje znatno brže treniranje na modernim GPU jedinicama [10].



Slika 3: Koder - dekode [10]

3.3.2. *Attention* mehanizam

Attention mehanizam omogućuje modelu da pri obradi svakog elementa sekvence obrati pažnju na relevantne dijelove ulaza. Funkcionira tako da za svaki element (upit) izračuna koliko je povezan sa svim ostalim elementima (ključevima), a zatim te povezanosti koristi kao težine za kombiniranje vrijednosti. Matematički se to izražava sljedećom formulom [10]:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Q (Query) predstavlja trenutni element koji se obrađuje, K (Key) su svi elementi ulaza na koje se može obratiti pažnja, a V (Value) su vrijednosti povezane s tim elementima. Normalizacija dijeljenjem s $\sqrt{d_k}$ osigurava stabilnost gradijenata tijekom treniranja [10].

Višeglavi *attention* (*multi-head attention*) proširuje ovaj mehanizam izvođenjem više paralelnih *attention* operacija, svaka s različitim naučenim projekcijama. Rezultati se zatim spajaju, što modelu omogućuje istovremeno fokusiranje na različite vrste informacija – primjerice, jedna “glava” može pratiti sintaktičke odnose, dok druga prati semantičke veze [10].

3.4. Rad s LLM-ovima u aplikacijama: tokeni, kontekst i kontrola generiranja

Veliki jezični modeli obrađuju tekst na način koji se razlikuje od tradicionalnog pristupa čitanju i pisanju. Umjesto da rad s tekstom temelji na riječima ili rečenicama, LLM-ovi koriste tokene kao osnovne jedinice obrade i generiraju tekst sekvencijalno, token po token. Razumijevanje ovih procesa ključno je za učinkovitu primjenu LLM-ova u praktičnim aplikacijama.

3.4.1. Tokeni i tokenizacija

Token predstavlja osnovnu jedinicu teksta koju model obrađuje tijekom svog rada. Ovisno o postupku tokenizacije, token može biti znak, podriječ, simbol ili cijela riječ [10]. Tokenizacija je bitna faza predobrade koja tekst rastavlja na te nedjeljive jedinice. Najčešće korištene metode tokenizacije kod LLM-ova uključuju *wordpiece*, kodiranje parom bajtova (BPE) i *unigramLM* [14].

Proces tokenizacije započinje tako što se ulazni tekst pretvara u niz tokena, pri čemu svaki token dobiva jedinstveni identifikator iz rječnika modela. Na primjer, riječ “tokenization” može biti podijeljena na više tokena poput “token”, “ization” ili čak na manje jedinice ovisno o primijenjenom algoritmu. Vaswani i suradnici koriste naučene vektorske reprezentacije (*embeddings*) za pretvaranje ulaznih i izlaznih tokena u vektore dimenzije d_{model} [10]. Ove vektorske reprezentacije omogućuju modelu da razumije semantičke odnose između različitih tokena.

Model generira izlaz sekvencijalno, predviđajući sljedeći token na temelju prethodno generiranih tokena. Ovaj proces iterativnog generiranja znači da model mora za svaki novi token izvršiti proračun kroz sve svoje slojeve, što izravno utječe na brzinu izvođenja i cijenu korištenja modela. Veći broj tokena u ulazu ili izlazu povećava računske zahtjeve i vrijeme potrebno za dobivanje odgovora. Različiti LLM-ovi koriste različite veličine rječnika tokena – primjerice, GPT-3 koristi približno 50.000 tokena, dok BLOOM koristi 250.000 tokena u svom rječniku [14].

3.4.2. Kontekst modela

Kontekst modela označava maksimalnu količinu teksta s kojom model može raditi odjednom. Može se zamisliti kao radna memorija modela – sve što model “vidi” i “pamti” tijekom obrade jednog zahtjeva ograničeno je veličinom ovog kontekstnog prozora. LLM-ovi su tradicionalno trenirani s ograničenim kontekstnim prozorima zbog računske složenosti mehanizma pažnje i visokih zahtjeva za memorijom [14].

Ograničenja konteksta predstavljaju važan praktični izazov. Model koji je treniran s ograničenom duljinom sekvence obično ne može uspješno generalizirati na veće duljine tijekom primjene. Proširivanje kontekstnog prozora tijekom finog podešavanja modela je spor, neučinkovit i računski skup proces [14]. Zbog toga su razvijene različite tehnike za proširenje konteksta poput interpolacije pozicijskih kodiranja i učinkovitijih mehanizama pažnje.

Gusti mehanizam globalne pažnje jedan je od glavnih uzroka ograničenja pri treniranju modela s većim kontekstnim prozorima. Korištenje učinkovitijih varijanti pažnje, poput lokalne, rijetke ili dilatirane pažnje, može značajno smanjiti računsku složenost. Na primjer, LongT5 koristi prijelaznu globalnu pažnju (TGlobal) koja kombinira lokalnu i globalnu pažnju, što omogućuje proširenje konteksta na 16.000 tokena [14].

Ograničenja konteksta motiviraju razvoj različitih tehnika za rad s dugim dokumentima. Tehnike poput sažimanja sadržaja, dijeljenja teksta na manje dijelove (*chunking*) i pristupa proširene generiranja s dohvaćanjem (RAG) omogućuju modelima pristup većoj količini informacija nego što stane u njihov kontekstni prozor. RAG pristup kombinira vanjsko pohranjivanje informacija s mogućnostima generiranja LLM-a, čime se omogućuje pristup ažurnim podacima i smanjuje pojava halucinacija [14].

3.4.3. Parametri generiranja: *temperature*, *top-k* i *top-p*

Nakon što model procesira ulazne tokene kroz svoje slojeve, dobiva se izlazna raspodjela vjerojatnosti za sve moguće sljedeće tokene. Transformer arhitektura koristi linearnu transformaciju i *softmax* funkciju za pretvaranje izlaza dekodera u vjerojatnosti sljedećeg tokena [10]. *Softmax* funkcija normalizira rezultate tako da zbroj svih vjerojatnosti bude jednak jedan, čime se dobiva valjana vjerojatnosna raspodjela.

Različiti parametri kontroliraju kako model bira sljedeći token iz ove raspodjele vjerojatnosti, što omogućuje podešavanje ravnoteže između kreativnosti i dosljednosti generiranog teksta.

Temperature je parametar koji kontrolira stupanj stohastičnosti u procesu generiranja. Temperature modificira vjerojatnosnu raspodjelu prije primjene *softmax* funkcije tako što dijeli logite s vrijednošću temperature. Niže vrijednosti temperature (bliže nuli) čine raspodjelu oštrijom – model postaje konzervativniji i češće bira tokene s najvećom vjerojatnosti, što rezultira predvidljivijim i stabilnijim odgovorima. Više vrijednosti temperature (veće od jedan) zaglađuju raspodjelu vjerojatnosti, dajući više šanse tokenima s nižom vjerojatnosti, što potiče veću raznolikost i kreativnost u generiranom tekstu. Temperatura se često koristi pri treniranju modela za generiranje prirodnog jezika kako bi se poboljšala kvaliteta prijevoda ili generiranih sekvenci [15].

Top-k uzorkovanje ograničava izbor sljedećeg tokena na k najvjerojatnijih kandidata iz raspodjele vjerojatnosti. Nakon što se identificira prvih k tokena s najvećom vjerojatnosti, model uzorkuje sljedeći token samo iz ovog ograničenog skupa, zanemarujući sve ostale mogućnosti. Ovaj pristup sprječava model da bira vrlo nevjerojatne tokene koji mogu narušiti koherentnost teksta, ali istovremeno omogućuje određenu raznolikost među najvjerojatnijim opcijama. Vrijednost parametra k određuje koliko će kandidata biti razmatrano – manja vrijednost rezultira konzervativnijim odabirom, dok veća vrijednost dopušta više varijabilnosti [15].

Top-p uzorkovanje, poznato i kao *nucleus sampling*, predstavlja dinamičniji pristup filtriranju kandidata. Umjesto fiksnog broja tokena, ovaj metoda odabire najmanji skup tokena čija kumulativna vjerojatnost prelazi zadani prag p (obično između 0.9 i 0.95). To znači da

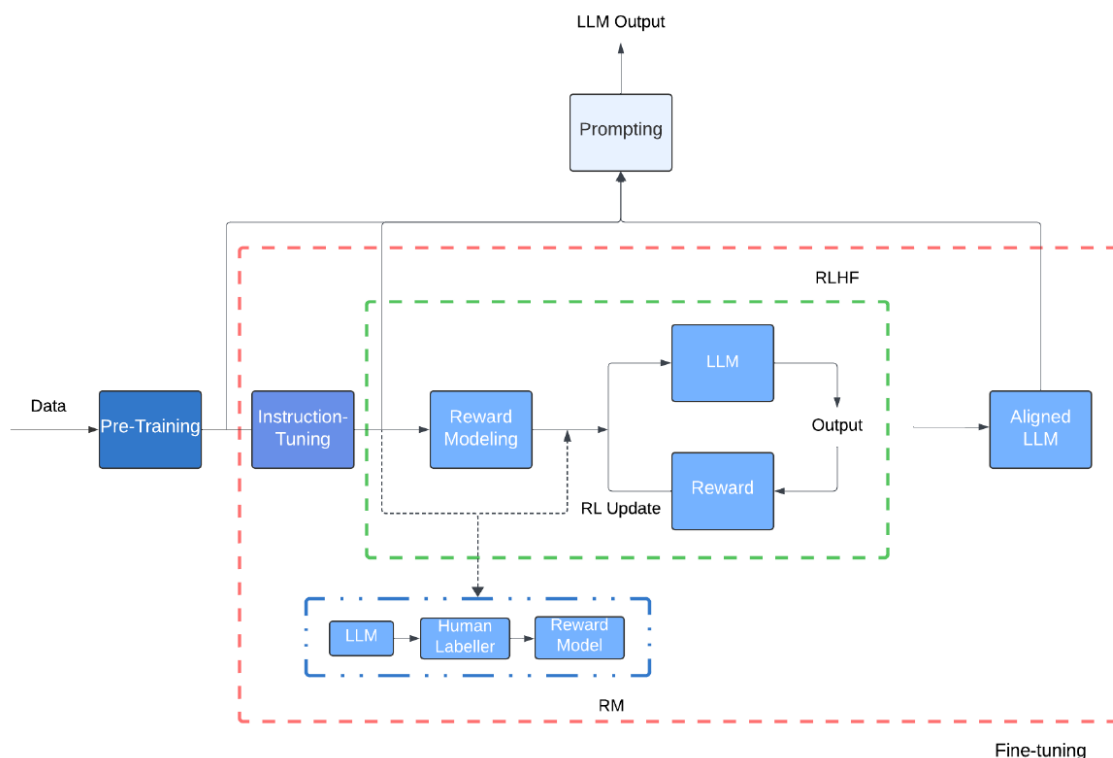
model automatski prilagođava veličinu skupa kandidata ovisno o distribuciji vjerojatnosti – kada je jedan token dominantan, skup će biti manji, a kada je više tokena približno jednako vjerojatno, skup će biti veći. Ovaj pristup omogućuje modelu da prilagodi razinu konzervativnosti ovisno o pouzdanosti predikcije u svakom koraku generiranja [15].

Kombinacija ovih parametara omogućuje fino podešavanje ponašanja modela prema specifičnim potrebama primjene. Za zadatke koji zahtijevaju preciznost i dosljednost, poput generiranja programskog koda ili formalnih dokumenata, koriste se niže vrijednosti temperature i konzervativnije strategije uzorkovanja. Za kreativne zadatke poput pisanja priča ili generiranja ideja, preporučuju se više vrijednosti temperature i fleksibilnije strategije uzorkovanja koje potiču raznolikost u generiranom sadržaju.

3.5. Prilagodba ponašanja bez treniranja: prompt engineering i *in-context* učenje

Tradicionalni pristup prilagodbi jezika modela za specifične zadatke ili domene zahtijeva postupak finog podešavanja, pri čemu se parametri prethodno treniranog modela ažuriraju korištenjem zadataka specifičnih podataka. Iako ovaj pristup omogućava značajno poboljšanje performansi za ciljne aplikacije, proces je računski skup i zahtijeva pristup kvalitetnim označenim podacima za svaki novi zadatak. Fino podešavanje može dovesti do katastrofičnog zaboravljanja (*catastrophic forgetting*), gdje model gubi znanje stečeno tijekom inicijalnog treniranja [15]. Zbog toga se razvio drugačiji pristup: umjesto da se mijenja model, mijenja se način na koji mu se postavljaju pitanja.

Veliki jezični modeli pokazuju sposobnost adaptacije putem promjena u načinu postavljanja upita, odnosno kroz tehniku koja se naziva *prompting* ili *prompt engineering*. Kao što je prikazano na Slici 4, *prompting* predstavlja metodu postavljanja upita treniranim LLM-ovima koja omogućava prilagodbu modela bez finog podešavanja parametara. Ova neočekivana sposobnost velikih jezičnih modela omogućava korisnicima da usmjeravaju ponašanje modela isključivo kroz pažljivo oblikovane tekstualne upute i primjere unutar samog upita. Dva ključna pristupa prilagodbi bez treniranja su *prompt engineering*, koji se fokusira na strukturiranje i formuliranje upita, te *in-context* učenje, pri čemu model uči iz demonstracija prisutnih u promptu [15], [16].



Slika 4: Dijagram toka različitih faza razvoja LLM-a od pred-treniranja do korištenja. RL označava *reinforcement learning*, RM *reward modeling*, a RLHF *reinforcement learning with human feedback* [12]

3.5.1. Prompt engineering

Prompt engineering predstavlja disciplinu oblikovanja uputa za velike jezične modele s ciljem postizanja konzistentnijih i kvalitetnijih rezultata. Ovaj pristup omogućava prilagodbu ponašanja modela bez potrebe za dodatnim treniranjem ili izmjenom parametara. Prompt može sadržavati različite elemente poput definiranja uloge modela, specificiranja željenog formata izlaza, postavljanja pravila ponašanja ili davanja kontekstualnih informacija [17].

Istraživanja pokazuju kako kvaliteta prompta značajno utječe na performanse modela. Dobro strukturiran prompt uključuje jasne i detaljne upute, može sadržavati pozitivne i negativne primjere željenog ponašanja te specificirati očekivanu duljinu ili format odgovora. Prompting može uključivati tehnike poput poticanja korak-po-korak rasuđivanja, gdje se model navodi da eksplicitno prikaže svoj proces razmišljanja prije davanja konačnog odgovora [14].

Prompt engineering postaje posebno važan u kontekstu *zero-shot* učenja, gdje model treba odgovoriti na zadatak koji nije vidio tijekom treniranja. Veliki jezični modeli pokazuju sposobnost *zero-shot* učenja, što znači da mogu odgovarati na upite bez ikakvog prethodnog primjera u promptu. Ova sposobnost omogućava modelima generalizaciju na nove zadatke isključivo na temelju jasno formuliranih uputa [14].

3.5.2. Sistemski i korisnički prompt

Moderni sustavi velikih jezičnih modela koriste distinkciju između sistemskog i korisničkog prompta kako bi omogućili fleksibilnije i kontroliranije interakcije. Sistemski prompt definira opće ponašanje modela kroz cijelu interakciju i postavlja osnovne smjernice za način rada. Ovaj prompt obično ostaje konstantan tijekom sesije i određuje kontekst u kojem model interpretira korisničke zahtjeve. Sistemski prompt može uključivati informacije o ulozi modela, stilskim preferencijama, ograničenjima u ponašanju ili specifičnim domenskim znanjima [16].

Korisnički prompt predstavlja konkretan zahtjev ili upit koji korisnik postavlja u određenom trenutku. Ovaj prompt je dinamičan i mijenja se s svakim zahtjevom korisnika. Model kombinira informacije iz sistemskog i korisničkog prompta kako bi generirao odgovarajući odgovor. Ova arhitektura omogućava održavanje konzistentnog ponašanja modela (kroz sistemski prompt) dok istovremeno omogućava fleksibilnost u odgovaranju na raznolike korisničke upite [14].

Važnost ove distinkcije posebno dolazi do izražaja u aplikacijama gdje je potrebno održavanje određenog stila komunikacije ili poštivanje specifičnih ograničenja kroz cijelu konverzaciju. Na primjer, sistemski prompt može definirati da model uvijek daje odgovore u određenom profesionalnom tonu ili da izbjegava određene teme, dok korisnički promptovi postavljaju konkretna pitanja unutar tog okvira.

3.5.3. *In-context* learning

In-context learning ili učenje u kontekstu predstavlja ključnu sposobnost velikih jezičnih modela koja im omogućava prilagodbu novim zadacima bez izmjene parametara modela. Ovaj fenomen se manifestira kroz sposobnost modela da uči iz primjera koji su dani unutar samog prompta. Kada se modelu pruži nekoliko primjera ulazno-izlaznih parova za određeni zadatak, model može generalizirati taj obrazac i primijeniti ga na nove ulaze [18].

Few-shot learning, kao specifičan oblik *in-context* učenja, pokazuje se posebno efektivnim. Istraživanja na GPT-3 modelu pokazala su kako performanse modela u *few-shot* scenariju premašuju performanse u *zero-shot* scenariju, što sugerira da veliki jezični modeli funkcioniraju kao meta-učenici (*meta-learners*) - modeli koji uče kako učiti iz malog broja primjera [18]. Broj primjera koji se daju modelu može varirati, ali čak i jedan ili dva dobro odabrana primjera mogu značajno poboljšati performanse na novom zadatku.

Format *in-context* učenja tipično uključuje strukturirane demonstracije gdje se svakom primjeru prilaže ulaz i očekivani izlaz. Model zatim koristi ove demonstracije kao reference za generiranje odgovora na novi upit. Kvaliteta i relevantnost odabranih primjera značajno utječu na performanse modela. Istraživanja pokazuju da primjeri trebaju biti što sličniji ciljanom zadatku te da redoslijed primjera također može utjecati na rezultate [14].

Ova tehnika omogućava privremeno usmjeravanje modela bez potrebe za skupim postupkom finog podešavanja. Parametri modela ostaju nepromijenjeni, a prilagodba se postiže isključivo kroz sadržaj prompta. To čini *in-context* učenje izuzetno praktičnim pristupom za brzu

prilagodbu modela različitim domenama i zadacima, posebno u scenarijima gdje je dostupno samo nekoliko primjera ili gdje je potrebna česta promjena zadataka [14].

In-context učenje ima svoja ograničenja. Svi primjeri moraju stati u kontekstni prozor modela, što ograničava broj demonstracija koje se mogu pružiti. Performanse ovog pristupa ovise o veličini modela - veći modeli pokazuju značajno bolje sposobnosti *in-context* učenja nego manji modeli [18].

3.6. Prilagodba modela treniranjem i optimizacija primjene

Veliki jezični modeli treniraju se na ogromnim količinama tekstualnih podataka kako bi stekli opće jezične sposobnosti, no njihova primjena u stvarnim sustavima često zahtijeva dodatne korake optimizacije. Prilagodba modela specifičnim domenama ili zadacima, smanjenje računalnih zahtjeva i usklađivanje s ljudskim preferencijama ključni su izazovi pri integraciji LLM-ova u proizvodne sustave. Dok su tehnike poput *prompt engineeringa* i *in-context* učenja korisne za prilagodbu ponašanja bez izmjene parametara modela, postoje scenariji u kojima je potrebno izravno modificirati sam model kako bi se postigla stabilnija i učinkovitija izvedba. Ovaj pristup omogućuje optimizaciju modela za specifične zadatke uz kontrolu nad resursnim zahtjevima i usklađenost s očekivanim ponašanjem [12].

Tehnike prilagodbe modela kreću se od potpunog ponovnog treniranja na domenskim podacima do metoda koje selektivno modificiraju mali podskup parametara, kao i postupaka kompresije koji smanjuju veličinu modela uz minimalan gubitak performansi. Fino podešavanje omogućuje specijalizaciju modela za određene zadatke ili domene, dok *parameter-efficient* metode postižu slične rezultate uz znatno manje računalnih resursa. Tehnike kao što su fino podešavanje usklađuju izlaze modela s ljudskim vrijednostima i preferencijama, što je ključno za primjenu u stvarnim sustavima gdje je sigurnost i korisnost prioritet. Metode kvantizacije i destilacije pak omogućuju razvoj manjih, bržih verzija modela pogodnih za implementaciju na uređajima s ograničenim resursima, čime se proširuje dostupnost i praktična primjenjivost LLM tehnologije [12].

3.6.1. Fino podešavanje i PEFT

Fino podešavanje predstavlja proces dodatnog treniranja prethodno naučenog jezičnog modela na skupu podataka specifičnom za određeni zadatak ili domenu. Fino podešavanje omogućuje modelu da specijalizira svoje znanje za uže definirane aplikacije. Ovaj proces nadograđuje postojeće reprezentacije naučene tijekom pretreniranja, prilagođavajući ih posebnostima ciljane domene ili zadatka. Kod tradicionalnog finog podešavanja ažuriraju se svi parametri modela, što može rezultirati stabilnijim i pouzdanijim ponašanjem u specifičnim scenarijima, ali zahtijeva značajne računalne resurse i memoriju proporcionalnu veličini modela [12].

Tradicionalni pristup finom podešavanju, iako učinkovit, suočava se s praktičnim izazovima kada se primjenjuje na moderne LLM-ove koji sadrže milijarde parametara. Ažuriranje svih parametara tijekom treniranja zahtijeva pohranu gradijenata i optimizacijskih stanja

za svaki parametar, što multiplicira memorijske zahtjeve i čini proces računalno zahtjevnim. Potpuno fino podešavanje nosi rizik od “katastrofičkog zaboravljanja” gdje model gubi opće jezične sposobnosti naučene tijekom pretriranja dok se prilagođava novom zadatku. Ovi izazovi motivirali su razvoj pristupa koji omogućuju učinkovitu prilagodbu modela uz minimiziranje računalnih i memorijskih troškova [12].

Parameter-Efficient Fine-Tuning (PEFT) metode predstavljaju alternativni pristup koji postiže prilagodbu modela modificirajući samo mali podskup parametara ili dodavanjem dodatnih adaptivnih komponenti. Jedna od najpoznatijih PEFT tehnika je LoRA (*Low-Rank Adaptation*) koja umjesto ažuriranja svih težina matrica u modelu, dodaje male podešavane matrice niskog ranga koje se kombiniraju s originalnim zamrznutim težinama. Ovaj pristup smanjuje broj parametara koje je potrebno trenirati - tipično za nekoliko redova veličine - što rezultira znatno nižim memorijskim zahtjevima i bržim treniranjem, uz zadržavanje kvalitete prilagodbe usporedive s potpunim finim podešavanjem. PEFT metode također omogućuju zadržavanje više različitih adaptacija istog osnovnog modela za različite zadatke, što je praktično u scenarijima gdje je potrebno podržati više specijaliziranih aplikacija [12].

3.6.2. Preference fine-tuning

Preference fine-tuning predstavlja naprednu tehniku koja omogućuje usklađivanje ponašanja jezičnih modela s ljudskim vrijednostima i preferencijama kroz učenje iz komparativnih ocjena. *Preference fine-tuning* koristi podatke u obliku usporedbi između različitih generiranih odgovora na isti upit. Ljudski korisnici ocjenjuju koji od ponuđenih odgovora je korisniji, sigurniji, informativniji ili prikladniji, a model uči da preferira generiranje odgovora koji su bolje ocijenjeni. U praksi se pokazalo da je ljudima lakše reći koji od dva odgovora je bolji nego opisati što idealan odgovor uopće jest, što ga čini pogodnim za domene gdje je kvaliteta subjektivna ili je teško uspostaviti formalizam [12].

Najpoznatiji pristup preference fine-tuningu je *Reinforcement Learning from Human Feedback* (RLHF) koji kombinira *supervised learning* s metodama učenja potkrepljivanjem (engl. *reinforcement learning*). Proces započinje skupljanjem usporednih ocjena ljudi gdje anotatori rangiraju različite odgovore modela na iste upite. Iz tih ocjena trenira se *reward* model koji uči predvidjeti ljudske preferencije, tj. procjenjuje koja vrsta odgovora će biti pozitivno ocijenjena. Konačno, koristeći algoritme poput *Proximal Policy Optimization* (PPO), osnovni jezični model se optimizira da maksimizira očekivanu nagradu prema naučenom *reward* modelu, efektivno učeći generirati odgovore usklađene s ljudskim preferencijama. Ovaj pristup pokazao se ključnim u razvoju dijaloških AI sustava poput ChatGPT, omogućivši modele koji ne samo da generiraju koherentne odgovore, već su također korisni i pružaju odgovore u stilu koji korisnici preferiraju [16].

Iza toga stoji jednostavan problem: model može biti tehnički sposoban, ali to ne znači da će se ponašati onako kako ljudi očekuju. Kroz iterativni proces prikupljanja povratnih informacija i učenja iz njih, modeli postaju sve prikladniji za interakciju s ljudima u stvarnim aplikacijama. Ovaj pristup omogućuje korekciju neželjenih ponašanja poput generiranja štetnog sadržaja, nedosljednosti u odgovorima ili neprimjerenog tona, što je teško postići samo kroz

supervised learning. *Preference fine-tuning* omogućava kontinuiranu prilagodbu modela kako se ljudske preferencije i društveni standardi mijenjaju tijekom vremena, čineći sustave fleksibilnijima i odgovornijima u dugoročnoj primjeni [12].

3.6.3. Kvantizacija i destilacija

Model od 70 milijardi parametara u punoj preciznosti zahtijeva više od 140 GB memorije, što premašuje kapacitet gotovo svake potrošačke grafičke kartice. Kvantizacija rješava taj problem smanjujući preciznost pohrane težina: umjesto 32-bitnog zapisa po parametru, koristi se 8-bitni ili čak 4-bitni, čime se memorijski zahtjev smanjuje četiri, odnosno osam puta uz relativno skroman pad kvalitete. Osim smanjenja memorijskog otiska, kvantizacija također ubrzava izvođenje jer operacije s nižom preciznošću se može brže izvršavati, posebno na specijaliziranom hardveru. Moderna istraživanja pokazuju kako je moguće kvantizirati veliki dio parametara LLM-a bez značajnog gubitka performansi, zahvaljujući redundanciji u naučenim reprezentacijama. Postoje različiti pristupi kvantizaciji, uključujući *post-training* kvantizaciju gdje se već trenirani model komprimira, te *quantization-aware training* gdje se kvantizacija uzima u obzir tijekom samog procesa treniranja [12].

Kvantizacija smanjuje preciznost postojećeg modela; destilacija ide korak dalje i stvara fundamentalno manji model. Ideja je jednostavna: umjesto da mali model uči samo iz označenih podataka, uči i iz distribucije odgovora većeg modela, tzv. mekih predikcija koje nose više informacije nego gola oznaka točnog odgovora. Na primjer, pri generiranju sljedeće riječi, *teacher* model može dodijeliti vjerojatnost 0.7 jednoj riječi, 0.2 drugoj i 0.1 trećoj - ove relativne vjerojatnosti pružaju *student* modelu uvid u nijanse jezika koje je *teacher* model naučio [12].

Primijenjene zajedno, ove dvije tehnike daju model koji je i manji i brži od originala, dovoljno kompaktan da radi lokalno na laptopu ili mobitelu, bez potrebe za slanjem upita u cloud. Upravo to je razlog zašto modeli poput Phi-3 ili Gemma 3 mogu postići rezultate usporedive s mnogo većim modelima, uz djelić njihovih računalnih zahtjeva [12].

3.7. RAG: povezivanje modela s vanjskim znanjem

Jezični model ne može znati što se promijenilo nakon što je treniran, i neće to ni naznačiti, nego će generirati uvjerljivo zvučeći odgovor koji je činjenično zastario ili netočan, što se naziva halucinacijom. Za sustav koji odgovara na pitanja iz vojnog zakonodavstva, to nije akademski problem: propis koji je bio na snazi u trenutku treniranja možda je izmijenjen. *Retrieval-Augmented Generation* (RAG) pristup rješava ova ograničenja integrirajući generativne sposobnosti velikih jezičnih modela s preciznošću dohvaćanja informacija u stvarnom vremenu, čime se omogućuje da modeli utemeljuju svoje odgovore u dinamički ažuriranim i provjerljivim izvorima [19].

3.7.1. Osnovna ideja RAG-a

Retrieval-Augmented Generation (RAG) predstavlja okvir koji spaja mehanizme dohvaćanja dokumenata temeljene na pretraživanju s generativnim modelima kako bi se poboljšala kvaliteta generiranih rezultata. RAG sustav koristi informacije koje su dohvaćene iz vanjskih izvora znanja, najčešće baza podataka, kako bi nadopunio proces generiranja odgovora. Ovaj pristup kombinira snagu velikih jezičnih modela u generiranju teksta s preciznošću dohvaćanja informacija u stvarnom vremenu, čime se znatno povećava točnost i pouzdanost odgovora [19].

Tijekom faze dohvaćanja (*retrieval*), sustav najprije konstruira upit na temelju korisničkog unosa koji je namijenjen jezičnom modelu. Ovaj upit se zatim koristi za pretraživanje naj-sličnijih ili najrelevantnijih dokumenata iz vanjskih izvora znanja. Mehanizam za dohvaćanje procjenjuje sličnost između korisničkog upita i vraćenih rezultata iz baze znanja kako bi odabrao najrelevantnije dokumente. Nakon što su najrelevantniji rezultati identificirani, proces obogaćivanja kombinira dohvaćene dokumente s izvornim korisničkim unosom kako bi se formirao konačni upit koji se proslijeđuje generativnom jezičnom modelu [19].

Veliki jezični modeli mogu imati ograničenu memoriju i zastarjele informacije, što dovodi do netočnih odgovora. Dohvaćanje relevantnih informacija iz vanjskog, ažuriranog skladišta omogućuje modelima da točno odgovaraju s referencama i koriste više informacija nego što je dostupno u njihovim parametrima. S obogaćivanjem dohvaćanja, manji modeli su pokazali izvedbu usporedivu s mnogo većim modelima - primjerice, model od 11 milijardi parametara može biti konkurentan modelu PaLM od 540 milijardi parametara, a model od 7,5 milijardi može dosegnuti izvedbu modela Gopher od 280 milijardi parametara [12].

3.7.2. Embeddings i vektorske baze podataka

Embeddings ili vektorske reprezentacije tekstualnih podataka predstavljaju temelj za semantičko pretraživanje u RAG sustavima. Mehanizmi za gusti dohvat (*dense retrievers*) koriste prethodno trenirane jezične modele kako bi transformirali i upite i dokumente u guste vektorske prikaze koji enkapsuliraju semantičko značenje teksta. Za razliku od rijetkih metoda pretraživanja (*sparse retrievers*) poput BM25 i TF-IDF koje se temelje na usporedbi ključnih riječi, guste metode omogućuju identifikaciju kontekstualno relevantnih dokumenata čak i u odsustvu točnog podudaranja riječi [19].

Primjer gustog mehanizma za dohvat je *Dense Passage Retriever* (DPR) koji koristi dvije neovisne, prethodno trenirane BERT mreže kao gusti enkoder. Jedna mreža enkodira svaki ulazni upit u vektor realnih brojeva, dok druga enkodira svaki tekstualni odlomak u vektor istih dimenzija. Sličnost između upita i tekstualnog odlomka mjeri se unutarnjim produktom ovih vektora, a enkoderi se treniraju maksimiziranjem unutarnjih produkata između vektora upita i odgovarajućih relevantnih odlomaka. Ovakvi enkoderi pokazuju robusnu izvedbu kao prethodno trenirani mehanizmi za dohvaćanje i široko se koriste u različitim RAG modelima [19].

Vektorske baze podataka omogućuju efikasno semantičko pretraživanje pohranjivanjem

i indeksiranjem vektorskih prikaza dokumenata. Kada korisnik postavi upit, sustav ga pretvara u vektorski oblik korištenjem istog modela za *embedding* i zatim pretražuje vektorsku bazu kako bi pronašao vektore koji su najbliži upitu u vektorskom prostoru. Vanjska memorija u RAG sustavima može biti održavana u različitim formatima kao što su dokumenti, vektori ili baze podataka, a neki sustavi održavaju posredničke (engl. *intermediate*) memorijske reprezentacije kako bi zadržali informacije kroz više iteracija [12].

3.7.3. Segmentacija dokumenata - *Chunking*

Nije svejedno kako se dokumenti dijele prije pohrane u vektorsku bazu. Segmentacija, ili *chunking*, može se provesti na razini tokena, rečenice, odlomka ili cijelog dokumenta, i taj odabir izravno utječe na to koliko će dohvaćeni sadržaj biti koristan modelu [19].

Gruba granularnost dohvaćanja (npr. dohvaćanje cijelih dokumenata) može pružiti širi kontekst, ali istovremeno može uvesti suvišan sadržaj koji potencijalno ometa jezični model u kasnijim (engl. *downstream*) zadacima. Fina granularnost dohvaćanja pruža preciznije informacije, ali može povećati složenost dohvaćanja i pohrane te ugroziti semantički integritet. Dohvaćanje na razini dokumenta funkcionira dobro kada zadatak zahtijeva sveobuhvatan kontekst, dok se dohvaćanje na razini odlomka (*chunk-level* i *passage-level*) fokusira na specifične odjeljke teksta, balansirajući kontekst s relevantnošću [19].

Odabir veličine i strategije segmentacije predstavlja važan dizajnerski izbor u RAG sustavima. Uspješni RAG modeli poput REALM, RAG i Atlas koriste pristup dohvaćanja na razini odlomka jer on pomaže u zadacima koji zahtijevaju specifične odgovore, pritom ne preopterećujući model sa suvišnim informacijama. Dohvaćanje na razini tokena, iako manje uobičajeno, može se koristiti za specijalizirane zadatke koji zahtijevaju točna podudaranja ili integraciju podataka izvan domene treniranja. Strategija segmentacije mora uzeti u obzir i ograničenja kontekstnog prozora modela - dijeljenje na manje cjeline omogućuje lakše pretraživanje i smanjuje rizik od prekoračenja maksimalne duljine ulaza [19].

3.8. GraphRAG

4. Implementacija chatbota

4.1. Opis domene i zahtjevi

4.2. Arhitektura sustava

4.3. Korišteni modeli i alati

4.4. Korpus i priprema podataka

5. Evaluacija chatbotova

Iskustvo s razvojem ovog sustava pokazalo je da točnost odgovora nije jedino što je važno, chatbot može citirati ispravan članak zakona, a opet biti nekoristan ako odgovor nije razumljiv ili primjenjiv. U kontekstu domene vojnog zakonodavstva, gdje pogreška u citiranju propisa ili pogrešna pravna interpretacija mogu imati ozbiljne posljedice, odabir primjerenih evaluacijskih metoda postaje od osobite važnosti. U ovom poglavlju opisuju se operativni aspekti inferencije i latencije, automatske metrike kvalitete teksta, pristup ljudskoj evaluaciji uz mjerenje međuoocjenjivačke pouzdanosti te pristup gdje jedan jezični model ocjenjuje izlaz drugog, tzv. LLM-as-a-Judge (*LLM-as-a-Judge*).

5.1. Operativni aspekti: inferenca i latencija

5.1.1. Online i batch inferenca

Ovisno o tome treba li odgovor odmah ili ne, razlikuju se dva pristupa pokretanju modela:

- **Online inferenca** isporučuje odgovore u stvarnom vremenu kao odgovor na svaki korisnički zahtjev. Primjerena je za interaktivne sustave u kojima korisnik očekuje neposredan odgovor, kao što je slučaj s chatbotovima koji odgovaraju na upite iz područja vojnog zakonodavstva.
- **Batch inferenca** obrađuje veći broj zahtjeva izvan realnog vremena, primjerice kao periodički noćni posao ili masovna obrada dokumenata. Pogodna je za scenarije u kojima latencija nije kritična, ali je propusnost (engl. *throughput*) važan čimbenik.

U ovom istraživanju koristi se isključivo *online* inferenca, budući da se evaluira interaktivni chatbot namijenjen odgovaranju na pitanja u stvarnom vremenu.

5.1.2. Latencija

Prag od 200 ms koji korisnici percipiraju kao “trenutačno” nije dostižan za LLM sustave koji generiraju dulje odgovore, ali nije ni relevantan cilj. Važnija je granica oko jedne sekunde: iznad nje korisnik svjesno čeka i gubi osjećaj dijaloga [20]. RAG konfiguracija u ovom sustavu dodaje korak dohvata prije generiranja, što povećava ukupnu latenciju, ali taj se trošak donekle kompenzira kraćim i preciznijim odgovorima koji ne zahtijevaju toliko generiranih tokena.

U sklopu evaluacije mjerena je ukupna latencija odgovora (u sekundama) za sve tri ispitivane konfiguracije sustava, standardnu (*vanilla*), konfiguraciju s dohvatom relevantnih dokumenata (*Retrieval-Augmented Generation*, RAG) i RAG s bazom znanja (GraphRAG) koja koristi Neo4j. Latencija RAG sustava očekivano je veća zbog dodatnog koraka dohvata i rekonstruktualizacije, no taj se trošak kompenzira potencijalno višom kvalitetom odgovora.

5.2. Automatske metrike

Ručna evaluacija 31 pitanja s tri evaluatora daje pouzdane, ali spore i skupe rezultate. Automatske metrike omogućuju da se isti posao napravi u sekundama, uz cijenu: ne znaju razlikovati pravno točan od pravno pogrešnog odgovora, čak i kad su formulacije gotovo identične. Iz tog razloga u ovom istraživanju automatske metrike služe kao pomoćne, ali ne i presudne metode prosudbe.

5.2.1. Leksičke metrike

Leksičke metrike mjere sličnost na razini podudaranja riječi i fraza između generiranog i referentnog odgovora.

5.2.1.1. ROUGE-L

Recall-Oriented Understudy for Gisting Evaluation (ROUGE) skup je metrika izvorno razvijenih za evaluaciju automatskog sažimanja teksta [21]. Varijanta **ROUGE-L** temelji se na najduljem zajedničkom podnizu (engl. *Longest Common Subsequence* – LCS) između generiranog i referentnog odgovora. LCS ne zahtijeva da se podudarni tokeni nalaze u neprekinutom slijedu, što ga čini robusnijim na varijacije u redoslijedu riječi. ROUGE-L vraća F1-mjeru kao harmonijsku sredinu preciznosti i opoziva.

U pravnoj domeni, parafraziranje nije uvijek vrlina. Ako chatbot umjesto “članka 45. stavka 2.” kaže “relevantnog dijela zakona”, semantički je blizu, ali korisnik koji želi citirati propis dobio je beskoristan odgovor. ROUGE-L kažnjava upravo takva odstupanja jer traži leksičko podudaranje, što ga čini prikladnijim za ovu domenu nego što bi bio u, recimo, evaluaciji kreativnog pisanja. Izostanak ili pogrešno navođenje referentnog članka kaznit će se niskim ROUGE-L rezultatom, čak i ako je ostatak odgovora semantički koherentan.

Napomena o metrici BLEU. Metrika *Bilingual Evaluation Understudy* (BLEU) [22] mjeri preciznost n-gramskog podudaranja i izvorno je razvijena za strojno prevođenje. U ovom istraživanju BLEU nije primijenjen kao primarna metrika iz dva razloga: pretjerano kažnjava legitimne parafrazirane odgovore koji su sadržajno točni, ne pruža informaciju o opozivu, što je u evaluaciji chatbota jednako važno kao i preciznost. ROUGE-L pokriva sličan leksički prostor uz bolja statistička svojstva za ovaj zadatak.

5.2.2. Semantičke metrike

Semantičke metrike procjenjuju sličnost na razini značenja, a ne samo podudaranja oblika riječi, što ih čini robusnijima na sinonime i parafraziranje.

5.2.2.1. BERTScore

BERTScore [23] semantička je metrika koja umjesto n-gramskog podudaranja koristi kontekstualne reprezentacije tokena dobivene predtreniranim BERT modelima. Za svaki token u generiranom odgovoru pronalazi se najbliži token u referentnom odgovoru (i obratno) prema kosinusnoj sličnosti njihovih vektora, a konačna ocjena agregira se u preciznost, opoziv i F1-mjeru.

U ovom istraživanju korišten je model `bert-base-multilingual-cased`, koji podržava hrvatski jezik i prikladan je za višejezične domene [23]. BERTScore je posebno koristan za hvatanje semantičke ekvivalentnosti kada chatbot koristi drugačiju formulaciju od ekspertovog referentnog odgovora, ali prenosi jednako značenje.

5.2.2.2. Kosinusna sličnost (Cosine Similarity)

Kosinusna sličnost mjeri kosinus kuta između dvaju vektora koji predstavljaju cijele odgovore (a ne pojedinačne tokene, kao BERTScore). Vrijednost 1 označava identičnu orijentaciju vektora (maksimalnu sličnost), vrijednost 0 označava da vektori nemaju ničeg zajedničkog u semantičkom prostoru.

Za pretvaranje odgovora u vektore korišten je model `intfloat/multilingual-e5-large` [24], koji je treniran na višejezičnom korpusu i pokazuje visoku učinkovitost na zadacima semantičkog pretraživanja i sličnosti u slavenskim jezicima. Dok BERTScore uspoređuje odgovore na razini tokena, kosinusna sličnost pruža globalnu procjenu tematske i kontekstualne podudarnosti cijelih odgovora.

5.2.3. Ograničenja automatskih metrika u pravnoj domeni

Unatoč prednostima skalabilnosti i reproducibilnosti, sve tri uporabljene automatske metrike dijele suštinsko ograničenje: pretpostavljaju visoku kvalitetu zlatnog standarda i ne detektiraju faktičke pravne pogreške.

Konkretno, chatbot može generirati odgovor koji je leksički i semantički blizak referentnom, ali koji pogrešno primjenjuje pravnu normu, primjerice, navodi ispravan članak zakona, ali s netočnim stavkom ili pogrešnom interpretacijom. Takav odgovor dobit će visoke automatske ocjene, ali je s pravnog stanovišta netočan.

Iz tog razloga automatske metrike u ovom istraživanju imaju *komplementarnu*, a ne primarnu ulogu: služe za brzo filtriranje i rangiranje odgovora, dok se konačna prosudba o pravnoj točnosti prepušta ljudskim evaluatorima i LLM ocjenjivaču s domenski specifičnim uputama.

5.3. Ljudska evaluacija

5.3.1. Evaluatori i dimenzije ocjenjivanja

U evaluaciji su sudjelovala tri evaluatora s komplementarnim domenskim znanjem: jedan pravni ekspert te dva vojna stručnjaka s iskustvom u primjeni vojnog zakonodavstva. Svaki evaluator radio je neovisno, bez uvida u ocjene ostalih, čime se minimizira efekt sidrenja (*anchoring bias*).

Za svako od $N = 31$ pitanja evaluatori su ocjenjivali odgovore chatbota na Likertovoj skali od 1 do 5 prema sljedećim dimenzijama:

Tablica 1: Dimenzije ljudske evaluacije i primarni evaluatori

Dimenzija	Opis	Primarni evaluator
Pravna točnost	Ispravnost citiranja zakona, članaka i propisa	Pravni ekspert
Citiranje propisa	Točnost navođenja specifičnih zakona i stavaka	Pravni ekspert
Praktična primjenjivost	Korisnost odgovora u stvarnoj vojnoj situaciji	Vojni stručnjaci
Potpunost odgovora	Obuhvaćenost svih relevantnih aspekata pitanja	Svi evaluatori
Jasnoća	Razumljivost i preciznost formulacije odgovora	Svi evaluatori

Skala ocjenjivanja definirana je kako slijedi: 1 – nedovoljno, 2 – slabo, 3 – prosječno, 4 – dobro, 5 – izvrsno.

5.3.2. Međuocjenjivačka pouzdanost

Pouzdanost ocjenjivanja kvantificirana je mjerenjem slaganja između dvojice vojnih evaluatora, koji su ocjenjivali iste odgovore neovisno. Primijenjene su dvije mjere:

- **Cohenov Kappa** (κ) [25] – mjera slaganja između dva ocjenjivača koja korigira slučajno slaganje. Vrijednosti se interpretiraju prema konvenciji: $\kappa < 0,20$ – slabo, $0,21-0,40$ – prihvatljivo, $0,41-0,60$ – umjereno, $0,61-0,80$ – dobro, $> 0,80$ – izvrsno [26].
- **Krippendorffov Alpha** (α) [27] – generalizacija koja je primjenjiva na ordinalne skale i više od dva ocjenjivača, što je relevantno za buduća proširenja evaluacijskog skupa.

Nakon neovisnog ocjenjivanja provedena je sesija usklađivanja (*adjudication session*) u kojoj su evaluatori raspravili slučajeve s najvećim odstupanjima ($\Delta \geq 2$), a konačne ocjene određene su konsenzusom.

5.4. LLM-as-a-Judge

5.4.1. Pristup i motivacija

Koncept *LLM-as-a-Judge* [28] rješava problem evaluacije upošljavanjem velikog jezičnog modela kao neovisnog suca koji evaluira izlaz drugog modela prema unaprijed zadanim kriterijima. LLM ocjenjivač može procjenjivati i kvalitativne aspekte poput koherentnosti argumentacije, primjerenosti tona i praćenja uputa.

LLM ocjenjivač uveden je iz tri razloga. Može obraditi puno više primjera nego što bi ikad stigli ručno, daje konzistentne ocjene jer uvijek koristi isti prompt, i bolje od leksičkih metrika prepoznaje parafrazirane ali sadržajno točne odgovore.

5.4.2. Implementacija i dizajn prompta

Kao LLM ocjenjivač korišten je model `claude-sonnet-4` (Anthropic). Odabir je motiviran visokom razinom praćenja uputa (*instruction following*) i strukturiranog izlaza u JSON formatu, što je ključno za automatsku obradu rezultata.

Sistemske prompt dizajniran je s eksplicitnim uputama za domenu vojnog zakonodavstva i zahtjevom za ocjenjivanje istih pet dimenzija kao i ljudski evaluatori (Tablica 1). Prompt uključuje:

- definiciju uloge (*role prompting*): evaluator je ekspert za vojno zakonodavstvo Republike Hrvatske,
- eksplicitan format izlaza: JSON objekt s numeričkim ocjenama 1–5 za svaku dimenziju i kratkim obrazloženjem na hrvatskom jeziku,
- referencu na zlatni standard: za svaki upit LLM-u se predaje i pitanje, i referentni odgovor eksperta, i odgovor chatbota,
- zabranu izlaza izvan JSON formata kako bi se izbjegla potreba za složenim parsiranjem teksta.

5.4.3. Poznata ograničenja

Primjena LLM-as-a-Judge pristupa donosi nekoliko inherentnih ograničenja na koja upozorava literatura [28], [29]:

- **Pristranost pozicije** (*position bias*): LLM ocjenjivači skloni su favorizirati odgovor koji se pojavljuje prvi u kontekstu. U ovom istraživanju odgovori se uvijek predaju u fiksnom redoslijedu (vanilla, RAG pa GraphRAG), što treba uzeti u obzir pri interpretaciji rezultata.

- **Pristranost samopohvale** (*self-enhancement bias*): modeli mogu favorizirati odgovore koji stilski nalikuju njihovim vlastitim odgovorima.
- **Ograničeno domensko znanje**: unatoč visokokvalitetnom modelu, LLM ocjenjivač može propustiti suptilne pravne pogreške koje bi iskusan pravni ekspert odmah prepoznao.
- **Neponovljivost**: stohastičnost jezičnih modela može uzrokovati marginalne razlike u ocjenama pri ponovljenom pokretanju s istim ulazom.

Iz navedenih razloga LLM-as-a-Judge tretira se kao *dopunska*, a ne zamjenska metoda u odnosu na ljudsku evaluaciju.

5.5. Obrazloženje odabira kombinacije metoda

U Tablici 2 prikazana je usporedba svih primijenjenih evaluacijskih metoda prema ključnim karakteristikama relevantnim za domenu vojnog zakonodavstva.

Tablica 2: Usporedba evaluacijskih metoda

Metoda	Skalabilnost	Semantičko razumijevanje	Detektira fakt. pogreške	Pravna specifičnost
ROUGE-L	visoka	nisko	ne	djelomično
BERTScore	visoka	visoko	ne	nisko
Cosine sličnost	visoka	visoko	ne	nisko
LLM-as-a-Judge	srednja	visoko	djelomično	visoko
Ljudska evaluacija	niska	visoko	da	visoko

Nijedna od ovih metoda sama za sebe nije dovoljna, svaka hvata nešto što druge propuštaju, pa ih je jedino smisleno koristiti zajedno. Automatske metrike (ROUGE-L, BERTScore, kosinusna sličnost) osiguravaju skalabilnu i reproducibilnu baznu liniju. LLM-as-a-Judge pruža kvalitativnu procjenu bez angažmana domenskih stručnjaka za svaki pojedinačni upit. Ljudska evaluacija s tri evaluatora i mjerenim međuocjenjivačkim slaganjem predstavlja zlatni standard čija je korelacija s automatskim metodama sama po sebi vrijedan doprinos istraživanju, jer ukazuje na primjenjivost automatskih metrika u ovoj specifičnoj domeni.

6. Statistička analiza rezultata evaluacije

6.1. Pregled prikupljenih podataka

Tablica 3: Deskriptivna statistika ljudskih ocjena po konfiguraciji

Konfiguracija	Dimenzija	$\bar{x}$	Med.	s	Min	Max
Vanilla	Pravna točnost	–	–	–	–	–
Vanilla	Citiranje propisa	–	–	–	–	–
Vanilla	Praktična primjenjivost	–	–	–	–	–
Vanilla	Potpunost odgovora	–	–	–	–	–
Vanilla	Jasnoća	–	–	–	–	–
RAG	Pravna točnost	–	–	–	–	–
RAG	Citiranje propisa	–	–	–	–	–
RAG	Praktična primjenjivost	–	–	–	–	–
RAG	Potpunost odgovora	–	–	–	–	–
RAG	Jasnoća	–	–	–	–	–
GraphRAG	Pravna točnost	–	–	–	–	–
GraphRAG	Citiranje propisa	–	–	–	–	–
GraphRAG	Praktična primjenjivost	–	–	–	–	–
GraphRAG	Potpunost odgovora	–	–	–	–	–
GraphRAG	Jasnoća	–	–	–	–	–

6.2. Usporedba konfiguracija: Vanilla vs. RAG vs. GraphRAG

Tablica 4: Kruskal-Wallisov test po dimenzijama ljudske evaluacije

Dimenzija	H	df	p
Pravna točnost	–	2	–
Citiranje propisa	–	2	–
Praktična primjenjivost	–	2	–
Potpunost odgovora	–	2	–
Jasnoća	–	2	–

Tablica 5: Post-hoc Wilcoxonov test s Bonferroni korekcijom (ljudska evaluacija)

Dimenzija	Par	W	p	$p_{\text{Bonf.}}$	r
Pravna točnost	Van. vs. RAG	–	–	–	–
	Van. vs. GraphRAG	–	–	–	–
	RAG vs. GraphRAG	–	–	–	–
Citiranje propisa	Van. vs. RAG	–	–	–	–
	Van. vs. GraphRAG	–	–	–	–
	RAG vs. GraphRAG	–	–	–	–
Praktična primjenjivost	Van. vs. RAG	–	–	–	–
	Van. vs. GraphRAG	–	–	–	–
	RAG vs. GraphRAG	–	–	–	–
Potpunost odgovora	Van. vs. RAG	–	–	–	–
	Van. vs. GraphRAG	–	–	–	–
	RAG vs. GraphRAG	–	–	–	–
Jasnoća	Van. vs. RAG	–	–	–	–
	Van. vs. GraphRAG	–	–	–	–
	RAG vs. GraphRAG	–	–	–	–

Tablica 6: Kruskal-Wallisov test i post-hoc usporedbe po automatskim metrikama

Metrika	Par	W	p	$p_{\text{Bonf.}}$	r
ROUGE-L	Van. vs. RAG	–	–	–	–
	Van. vs. GraphRAG	–	–	–	–
	RAG vs. GraphRAG	–	–	–	–
BERTScore	Van. vs. RAG	–	–	–	–
	Van. vs. GraphRAG	–	–	–	–
	RAG vs. GraphRAG	–	–	–	–
Kos. sličnost	Van. vs. RAG	–	–	–	–
	Van. vs. GraphRAG	–	–	–	–
	RAG vs. GraphRAG	–	–	–	–

6.3. Korelacija automatskih metrika s ljudskom evaluacijom

Tablica 7: Spearmanova korelacija (ρ) automatskih metrika s dimenzijama ljudske evaluacije

Dimenzija	ROUGE-L	BERTScore	Kos. sličnost
Pravna točnost	–	–	–
Citiranje propisa	–	–	–
Praktična primjenjivost	–	–	–
Potpunost odgovora	–	–	–
Jasnoća	–	–	–

6.4. Međuocjenjivačka pouzdanost

Tablica 8: Međuocjenjivačka pouzdanost između vojnih evaluatora

Dimenzija	κ	α	Interpretacija
Pravna točnost	—	—	—
Citiranje propisa	—	—	—
Praktična primjenjivost	—	—	—
Potpunost odgovora	—	—	—
Jasnoća	—	—	—

6.5. LLM-as-a-Judge: usporedba s ljudskom evaluacijom

Tablica 9: Spearmanova korelacija LLM ocjenjivača s ljudskom evaluacijom po konfiguraciji

Dimenzija	ρ (Vanilla)	ρ (RAG)	ρ (GraphRAG)
Pravna točnost	—	—	—
Citiranje propisa	—	—	—
Praktična primjenjivost	—	—	—
Potpunost odgovora	—	—	—
Jasnoća	—	—	—

6.6. Latencija

Tablica 10: Statistika latencije odgovora (u sekundama)

Konfiguracija	$\bar{x}$	Med.	P_{95}	s
Vanilla	—	—	—	—
RAG	—	—	—	—
GraphRAG	—	—	—	—

6.7. Sinteza rezultata

7. Zapažanja tijekom razvoja Chatbota

Razvoj RAG chatbota za hrvatsko obrambeno pravo prošao je kroz **deset** iteracija, pri čemu je svaka verzija uvodila specifične konfiguracije s ciljem poboljšanja točnosti i stabilnosti odgovora.

7.1. Evolucija RAG sustava kroz iterativno poboljšavanje

Početne verzije pokazale su osnovnu funkcionalnost, ali s niskom točnošću. Vanilla model postigao je samo 19% točnosti, dok je RAG s 42% demonstrirao potencijal pristupa temeljenog na dohvaćanju relevantnog konteksta. Treća verzija postigla je iznenađujućih 90% točnosti, ali naknadnom analizom se otkrilo kako je evaluacijski skup pitanja (zlatni standard) bio uključen u korpus kao Q&A parovi, što je rezultiralo curenjem podataka (*data leakage*) i umjetno visokim rezultatima. Ovaj nalaz naglasio je važnost jasne separacije trenažnih podataka od evaluacijskog skupa te potrebu za pažljivim dizajnom eksperimenata u domeni pravnih chatbotova.

7.2. Specifični tehnički izazovi i njihova rješenja

Najznačajniji tehnički problem kroz sve iteracije bio je multilingvalni “bleeding” bug, neželjeno generiranje teksta na bugarskom, srpskom ili ruskom jeziku, često u ćirilicom pismu. Problem je prvi put identificiran u verziji v06 gdje je u četiri slučaja odgovor završavao na bugarskom jeziku ili u ćirilici, primjerice: года. Этот срок указан в Статье 159. Uzrok je bio u činjenici da je EuroLLM multilingvalni model treniran na blisko srodnim južnoslavenskim jezicima, što u embedding prostoru stvara preklapanje između hrvatskog, srpskog i bugarskog. Pokušaji rješavanja ovog problema kroz prompt engineering, dodavanjem eksplicitnih uputa poput “NIKAD ne koristi ćirilicu, bugarski ili srpski jezik”, pokazali su se neučinkovitima, pa čak i kontraproduktivnima. Verzija v09 s agresivnim prompt upozorenjima rezultirala je paradoksalnim efektom: model je razvio svojevrsnu “samosvijest o uputama” (*meta-awareness*) i počeo citirati vlastite upute u izlazu, npr. УВАГА! Не користите ЋИРИЛИКУ или СРПСКИЕ ЕЗЫКИ, doslovno upozoravajući sam sebe na ukrajinskom jeziku.

7.3. Optimizacija generacijskih parametara i njihov utjecaj

Osim lingvističkih problema, iterativno testiranje otkrilo je kritičnu važnost pažljivog balansiranja generacijskih hiperparametara. Verzija v09 pokušala je povećati determinizam snižavanjem temperature s 0.7 na 0.3 kako bi se smanjila varijabilnost odgovora, što je rezultiralo katastrofalnim **no-space bugom**, model je generirao 500+ znakova bez razmaka (npr. “SustavimenovanjaprinadelnacenikavGlavnomstožeru”). Niska temperatura uzrokovala je da model ulazi u repetitivne uzorke iz kojih se, bez dovoljno stohastičnosti, nije mogao izvući. Istovremeno, parametar `repetition_penalty` morao je biti pažljivo kalibriran: vrijednost 0.6 u v06

bila je preniska što je dopuštalo ponavljanje rečenica, dok je vrijednost 1.3 u kasnijim verzijama postigla optimalan balans.

Kroz ove iteracije postalo je jasno da uspješan RAG sustav za pravnu domenu zahtjeva multimodalnu obranu od grešaka: kombinaciju prompt engineeringa za visokorazinsko usmjeravanje, post-processinga za sigurnosnu provjeru te pažljivo izbalansiranih generacijskih parametara koji omogućuju dovoljnu kreativnost za izbjegavanje loop-ova, ali dovoljnu konzistentnost za precizne pravne odgovore. Također se pokazalo da pojedinačne metrike poput BLEU-a ili ROUGE-a nisu dovoljan pokazatelj kvalitete, potrebna je kombinacija automatskih metrika (posebno BERTScore koji korelira s ručnim ocjenama), LLM-as-a-Judge evaluacije za skalabilnost, te ekspertne validacije za ground truth u pravnoj domeni.

7.4. Zaključak i naučene lekcije

Analiza iterativnog razvoja kroz deset verzija pokazala je da razvoj specijaliziranog chatbota za pravnu domenu nije linearan proces, već zahtijeva sistemski pristup dijagnostici i rješavanju problema. Ključna spoznaja je da **tehnička implementacija mora pratiti prompt engineering**, potrebno je duboko razumijevanje interakcije između modela, korpusa, prompt dizajna i generacijskih parametara. Iterativni pristup omogućio je identifikaciju problema koji nisu vidljivi u pojedinačnim testovima: od metodoloških grešaka poput curenja podataka, preko lingvističkih izazova multilingvalnog modela, do suptilnih efekata hiperparametara koji mogu uzrokovati potpuno nečitljive odgovore.

Najvažnija lekcija je da **prompt engineering sam za sebe nije dovoljno robustan**, pokazalo se da eksplicitne tekstualne zabrane mogu čak pogoršati problem stvarajući “samo-svijest o uputama” u modelu. Finalna implementacija nije u potpunosti primijenila sve identificirane korekcije. Ovo znači da su bugovi poput no-space greške i multilingvalnog “curenja” **i dalje mogući u produkcijskom okruženju**, unatoč identifikaciji uzroka.

Konačna verzija s pojednostavljenim system promptom postiže zadovoljavajuće rezultate u kontroliranim uvjetima, ali **nedostatak robusnih tehničkih zaštita** (post-processing validacija, optimalna temperatura) predstavlja rizik za produkcijsku upotrebu. Ovo potvrđuje glavnu tezu istraživanja: **za produkcijski RAG sustav u pravnoj domeni potreban je hibridni pristup** koji integrira prompt engineering za visokorazinsko usmjeravanje, tehničko filtriranje za kritične zabrane, post-processing za validaciju outputa, te pažljivo eksperimentalno podešavanje generacijskih parametara. Bez ovog integralnog pristupa, sustav ostaje podložan greškama koje mogu kompromitirati pouzdanost u pravnom kontekstu gdje je faktička točnost kritična.

8. Zaključak

Popis literature

- [1] J. Naisbitt, *Megatrends: 10 new directions transforming our lives*. Warner Books, 1982., str. 17.
- [2] M. Al-Amin, M. S. Ali, A. Salam i dr., „History of generative Artificial Intelligence (AI) chatbots: past, present, and future development,” *arXiv preprint arXiv:2402.05122*, 2024.
- [3] A. M. Turing, „Computing Machinery and Intelligence,” *Mind*, sv. 59, br. 236, str. 433–460, 1950.
- [4] R. Lane, A. Hay, A. Schwarz, D. M. Berry i J. Shrager, „ELIZA Reanimated: The world’s first chatbot restored on the world’s first time sharing system,” *arXiv preprint arXiv:2501.06707*, 2025.
- [5] D. F. Murad, A. G. Iskandar, E. Fernando, T. S. Octavia i D. E. Maured, „Towards smart LMS to improve learning outcomes students using LenoBot with natural language processing,” *2019 6th International Conference on Information Technology, Computer and Electrical Engineering (ICITACEE)*, IEEE, 2019., str. 1–6.
- [6] E. Adamopoulou i L. Moussiades, „An Overview of Chatbot Technology,” *Artificial Intelligence Applications and Innovations (AIAI 2020)*, serija IFIP Advances in Information and Communication Technology, sv. 584, Springer Nature Switzerland AG, 2020., str. 373–383. DOI: 10.1007/978-3-030-49186-4_31.
- [7] R. S. Wallace, „Artificial Intelligence Markup Language (AIML) Version 1.0.1,” A.L.I.C.E. AI Foundation, teh. izv., 2001.
- [8] S. Team, *Hey Siri: An On-device DNN-powered Voice Trigger for Apple’s Personal Assistant*, <https://machinelearning.apple.com/research/hey-siri>, [Pristupljeno 09.01.2026], 2017.
- [9] P. Muangkammuen i N. Intiruk, „Automated Thai-FAQ Chatbot using RNN-LSTM,” *2019 5th International Conference on Engineering, Applied Sciences and Technology (ICEAST)*, IEEE, 2019., str. 1–4. DOI: 10.1109/ICEAST.2019.8712781.
- [10] A. Vaswani, N. Shazeer, N. Parmar i dr., „Attention is all you need,” *Advances in neural information processing systems*, sv. 30, 2017.
- [11] T. T. Stoyanova i S. R. Stoyanov, „Chatbots: History, Technology and Application in Higher Education,” *Annual of Konstantin Preslavsky University of Shumen, Faculty of Mathematics and Informatics*, sv. XXIV C, str. 27–40, 2023. DOI: 10.46687/KNBU2020.

- [12] H. Naveed, A. U. Khan, S. Qiu i dr., „A comprehensive overview of large language models,” *ACM Transactions on Intelligent Systems and Technology*, sv. 16, br. 5, str. 1–72, 2025.
- [13] C. Staff, *What is an AI Engineer? (And how to become one)*, <https://www.coursera.org/articles/ai-engineer>, [Pristupljeno 13.01.2026], 2025.
- [14] C. Zhou, Q. Li, C. Li i dr., „A comprehensive survey on pretrained foundation models: A history from bert to chatgpt,” *International Journal of Machine Learning and Cybernetics*, sv. 16, br. 12, str. 9851–9915, 2025.
- [15] C. Huyen, *AI Engineering: Building Applications with Foundation Models*. Sebastopol, CA: O'Reilly Media, 2025., ISBN: 978-1-098-16630-4.
- [16] L. Ouyang, J. Wu, X. Jiang i dr., „Training Language Models to Follow Instructions with Human Feedback,” *arXiv preprint arXiv:2203.02155*, 2022.
- [17] E. Saravia, *Prompt Engineering Guide*, 12. 2022.
- [18] T. B. Brown, B. Mann, N. Ryder i dr., „Language Models are Few-Shot Learners,” *arXiv preprint arXiv:2005.14165*, 2020.
- [19] B. Han, T. Susnjak i A. Mathrani, „Automating Systematic Literature Reviews with Retrieval-Augmented Generation: A Comprehensive Overview,” *Applied Sciences*, sv. 14, br. 19, 2024., ISSN: 2076-3417. DOI: 10.3390/app14199103.
- [20] J. Nielsen, *Usability Engineering*. Academic Press, 1993.
- [21] C.-Y. Lin, „ROUGE: A Package for Automatic Evaluation of Summaries,” *Proceedings of the ACL Workshop on Text Summarization Branches Out*, 2004., str. 74–81.
- [22] K. Papineni, S. Roukos, T. Ward i W.-J. Zhu, „BLEU: a Method for Automatic Evaluation of Machine Translation,” *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002., str. 311–318.
- [23] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger i Y. Artzi, „BERTScore: Evaluating Text Generation with BERT,” *International Conference on Learning Representations*, 2020.
- [24] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder i F. Wei, „Multilingual E5 Text Embeddings: A Technical Report,” *arXiv preprint arXiv:2402.05672*, 2024.
- [25] J. Cohen, „A Coefficient of Agreement for Nominal Scales,” *Educational and Psychological Measurement*, sv. 20, br. 1, str. 37–46, 1960.
- [26] J. R. Landis i G. G. Koch, „The Measurement of Observer Agreement for Categorical Data,” *Biometrics*, sv. 33, br. 1, str. 159–174, 1977.
- [27] K. Krippendorff, „Computing Krippendorff’s Alpha-Reliability,” Annenberg School for Communication, University of Pennsylvania, teh. izv., 2011.
- [28] L. Zheng, W.-L. Chiang, Y. Sheng i dr., „Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” *Advances in Neural Information Processing Systems*, 2023.
- [29] P. Wang, L. Li, L. Chen i dr., „Large Language Models are not Fair Evaluators,” *arXiv preprint arXiv:2305.17926*, 2023.

Popis slika

1.	Originalni ispis koda ELIZA pronađen u Weizenbaumovim arhivama [5]	4
2.	Vremenska linija chatbota [11]	10
3.	Koder - dekode [10]	14
4.	Dijagram toka različitih faza razvoja LLM-a od pred-treniranja do korištenja. RL označava <i>reinforcement learning</i> , RM <i>reward modeling</i> , a RLHF <i>reinforcement learning with human feedback</i> [12]	18

Popis tablica

1.	Dimenzije ljudske evaluacije i primarni evaluatori	29
2.	Usporedba evaluacijskih metoda	31
3.	Deskriptivna statistika ljudskih ocjena po konfiguraciji	32
4.	Kruskal-Wallisov test po dimenzijama ljudske evaluacije	32
5.	Post-hoc Wilcoxonov test s Bonferroni korekcijom (ljudska evaluacija)	33
6.	Kruskal-Wallisov test i post-hoc usporedbe po automatskim metrikama	33
7.	Spearmanova korelacija (ρ) automatskih metrika s dimenzijama ljudske evaluacije	33
8.	Međuocjenjivačka pouzdanost između vojnih evaluatora	34
9.	Spearmanova korelacija LLM ocjenjivača s ljudskom evaluacijom po konfiguraciji	34
10.	Statistika latencije odgovora (u sekundama)	34

Prilozi

1. Prilog 1